**Technická univerzita v Košiciach**
**Fakulta elektrotechniky a informatiky**

# Mikro-tendre Studentskej rady - on-chain denník malých nákupov

**Bakalárska práca**

**2025** **Kostiantyn Ostrovskyi**

**Technická univerzita v Košiciach**
**Fakulta elektrotechniky a informatiky**

# Mikro-tendre Studentskej rady - on-chain denník malých nákupov

**Bakalárska práca**

| | |
|---|---|
| Študijný program: | Informatika |
| Študijný odbor: | Informatika |
| Školiteľ: | Ing. Daniel Šolc |
| Školiace pracovisko: | Katedra počítačov a informatiky (KPI) |

**Košice 2025** **Kostiantyn Ostrovskyi**

## Abstrakt v slovenčine

Táto práca predstavuje riešenie problému malých nákupov študentskej rady prostredníctvom vytvárania tendrov. Po analýze riešení konkurentov, konkrétne tradičných tendrových platforiem a systémov automatického riadenia financií, je zrejmé, že väčšina z nich používa príliš veľa nepotrebných nástrojov, ktoré sú príliš drahé a nevhodné pre študentské organizácie. Preto sa v tejto práci bude skúmať inovatívne riešenie — vytvorenie decentralizovanej aplikácie (dApp) na báze technológie blockchain.

## Kľúčové slová v slovenčine

LaTeX, programovanie, mikrotendre, študentská rada, blockchain, smart kontrakty, IPFS, transparentnosť nákupov, kvorum, decentralizácia, TUKE

## Abstrakt v angličtine

This thesis presents a solution to the problem of small purchases by student councils through the creation of tenders. After analyzing competitors' solutions, specifically traditional tender platforms and automated financial management systems, it can be concluded that most of them use too many unnecessary tools that are too expensive and unsuitable for student organizations. Therefore, this thesis will explore an innovative solution — the creation of a decentralized application (dApp) based on blockchain technology.

## Kľúčové slová v angličtine

LaTeX, programming, micro-tenders, student council, blockchain, smart contracts, IPFS, procurement transparency, quorum, decentralization, TUKE

## Bibliografická citácia

OSTROVSKYI, Kostiantyn. *Mikro-tendre Studentskej rady - on-chain denník malých nákupov*. Košice: Technická univerzita v Košiciach, Fakulta elektrotechniky a informatiky, 2025. 43 s. Vedúci práce: Ing. Daniel Šolc

Na túto stranu vložte zadávací list. To urobíte nasledovne:

1. Zadávací list naskenujte v odtieňoch sivej (čiernobielo) na 200 až 300 DPI.
2. Sken uložte do priečinku `figures/`.
3. V súbore `metadata.tex` zadajte umiestnenie súboru pomocou príkazu `\thesisspec{figures/zadavaci.list}`

**Do jednej kópie práce však nezabudnite vložiť originál zadávacieho listu!**

## Čestné vyhlásenie

Vyhlasujem, že som záverečnú prácu vypracoval samostatne s použitím uvedenej odbornej literatúry.

## Poďakovanie

Na tomto mieste by som sa rád poďakoval svojmu vedúcemu práce za jeho čas a odborné vedenie počas riešenia mojej záverečnej práce. Rovnako by som sa rád poďakoval svojim rodičom a priateľom za ich podporu a povzbudzovanie počas celého môjho štúdia.

# Obsah


**1 Úvod** **1**

**2 A little bit background** **4**

2.1 Existujúce systémy tendrov . . . . . 4

2.2 Tender system evaluation . . . . . 5

2.3 Blockchain technology . . . . . 5

2.3.1 General Working Sequence of Blockchain . . . . . 6

2.3.2 Transaction . . . . . 7

2.3.3 Composition of a Block . . . . . 7

2.3.4 Blockchain Platforms . . . . . 8

2.3.5 Smart-contracts . . . . . 10

2.3.6 Voting system . . . . . 11

2.4 IPFS . . . . . 11

2.4.1 Principles of IPFS operation . . . . . 12

2.4.2 Why IPFS for documents? . . . . . 13

2.4.3 Alternative . . . . . 13

2.4.4 Integration with Blockain . . . . . 15

**3 Requirements specification** **16**

3.1 Analysis of the current situation . . . . . 16

3.2 Problem identification . . . . . 17

3.3 Functional requirements . . . . . 18

3.3.1 Functional requirements for council members . . . . . 18

3.3.2 Functional requirements for vendors . . . . . 19

3.3.3 Functional requirements for the public . . . . . 20

3.4 Non-functional requirements . . . . . 20

3.4.1 Bezpečnosť . . . . . 20

3.4.2 Výkonnosť . . . . . 21

3.4.3 Použiteľnosť . . . . . 21

3.4.4 Spoľahlivosť a dostupnosť . . . . . 21
3.4.5 Transparentnosť . . . . . 21
3.5 Use cases . . . . . 22

**4 Návrh riešenia** **24**
4.1 Celková architektúra systému . . . . . 24
4.1.1 Prehľad architektúry . . . . . 24
4.1.2 Technologický stack . . . . . 26
4.1.3 Komunikácia medzi komponentmi . . . . . 27
4.1.4 Architektonické rozhodnutia . . . . . 28
4.2 Návrh smart kontraktu MicroTender . . . . . 29
4.2.1 Monolitická architektúra kontraktu . . . . . 29
4.2.2 Dátové štruktúry . . . . . 30
4.2.3 Roly a oprávnenia . . . . . 30
4.2.4 Stavový model tendra . . . . . 31
4.3 Životný cyklus tendra a proces obstarávania . . . . . 32
4.3.1 Etapa 1: Vytvorenie a publikovanie tendra . . . . . 32
4.3.2 Etapa 2: Registrácia dodávateľa a podanie ponuky . . . . . 33
4.3.3 Etapa 3: Hlasovanie a ukončenie tendra . . . . . 34
4.4 Návrh integrácie s IPFS . . . . . 35
4.4.1 Proces nahrávania cez Pinata . . . . . 35
4.4.2 Prístup k dokumentom . . . . . 36
4.5 Návrh používateľského rozhrania (UI/UX) . . . . . 36
4.5.1 UX výskum a konceptuálny model . . . . . 37
4.5.2 First prototype . . . . . 37
4.5.3 Používateľské testovanie prvého prototypu . . . . . 38
4.5.4 Druhý prototyp a jeho overenie . . . . . 39
4.5.5 Zmeny a vylepšenia oproti prvému prototypu . . . . . 39
4.5.6 Popis finálneho rozhrania . . . . . 40
4.5.7 Používateľské testovanie druhého prototypu . . . . . 40

**5 Vyhodnotenie** **41**

**6 Záver** **42**

**Zoznam príloh** **44**

# Zoznam obrázkov


2.1 Príklad architektúry blockchainového systému [**Rehmani2021**] . . . . 6
2.2 The structure of a Blockchain[**Liang2020**] . . . . . 7
2.3 Difference beetween IPFS and HTTP[**norman2026ipfsGateways**] . . . 11
3.1 Hlavní aktéri a jejich funkční požadavky v systému mikro-tendrov . . 18
3.2 Diagram prípadov použitia pre systém mikro-tendrov . . . . . 22
4.1 Architektúra systému MicroTender: prezentačná vrstva (React), logická vrstva (smart kontrakt) a dátová vrstva (blockchain + IPFS). . . . . 25
4.2 Stavový model tendra, ktorý zaručuje správne poradie fáz procesu obstarávania. . . . . 31
4.3 Vytvorenie a publikovanie tendra. . . . . 33
4.4 Registrácia dodávateľa a podanie ponuky. . . . . 34
4.5 Proces nahrávania dokumentu do IPFS. . . . . 36
4.6 Konceptuálny model používateľského rozhrania. . . . . 37
4.7 Ukážka z prvého prototypu dashboardu. . . . . 38

# Zoznam tabuliek

2.1 Porovnanie elektronických verejných obstarávacích systémov . . . . . 5
2.2 Hlavné oblasti použitia rôznych blockchainových systémov . . . . . 8
2.3 Comparative Overview of Blockchain Platforms (2025) . . . . . 9
2.4 Porovnanie protokolov HTTP a IPFS . . . . . 12
2.5 Porovnanie protokolov HTTP, IPFS, Filecoin a Arweave . . . . . 14
4.1 Porovnanie rolí v procese obstarávania a v smart kontrakte . . . . . 31

# 1 Úvod

Študentské rady vo vzdelávacích inštitúciách pravidelne nakupujú rôzne položky na podujatia, zásoby a iné účely. Sumy takýchto nákupov sú zvyčajne malé, ale kľúčovým aspektom, ktorý treba zvážiť, zostáva potreba systematického prístupu k účtovníctvu a kontrole. To často vedie k problémom, pretože je ťažké, ak nie nemožné, overiť transparentnosť takýchto transakcií. Za zmienku tiež stojí, že značný čas strávený rozhodovaním zostáva významnou výzvou.

Vzhľadom na všetky uvedené skutočnosti sa zdá pravdepodobné, že dobrým riešením by bolo použitie systému verejného obstarávania, ktorý dokáže tento problém efektívne riešiť. Toto skutočne môže byť dobré riešenie, ale rád by som navrhol zvážiť technicky pokročilejšiu verziu systému verejného obstarávania založeného na technológii blockchain s využitím inteligentných zmlúv vo forme decentralizovanej aplikácie (dApp). Medzi výhody použitia technológie blockchain patrí transparentnosť a nemennosť údajov, decentralizácia, sledovanie transakcií, automatizácia prostredníctvom inteligentných zmlúv, dôvera bez sprostredkovateľov a predovšetkým bezpečnosť. Tieto vlastnosti sú nevyhnutné, pokiaľ ide o finančné transakcie, ako v našom prípade.

Po analýze konkurenčných riešení, konkrétne konvenčných platforiem pre verejné obstarávanie a automatizovaných systémov finančného riadenia, vidíme, že väčšina z nich používa príliš veľa nepotrebných nástrojov, je príliš drahá a nevhodná pre študentské organizácie. Hoci blockchain nevyrieši každý problém a nie je svätým grálom, táto technológia ponúka nové možnosti využitia, ktoré nám určite uľahčia úlohu. Cieľom tejto práce je preto vytvoriť inovatívnu decentralizovanú aplikáciu, ktorá zabezpečí bezpečnosť, dôveru a efektívnosť v mikro-tendroch.

Práca sa zameriava nielen na implementáciu, ale aj na porovnanie jej výhod oproti konkurencii v reálnom prostredí, konkrétne pri použití študentskou radou na malé nákupy. Výsledná decentralizovaná aplikácia v konečnom dôsledku demonštruje, ako možno technológiu blockchain využiť na zlepšenie transparentnosti a efektívnosti finančného riadenia.

## Formulácia úlohy

Študentské rady zohrávajú dôležitú úlohu vo vnútornom živote univerzít. Pripravujú a organizujú podujatia, nakupujú materiály a ďalšie aktivity, z ktorých väčšina si vyžaduje finančné výdavky. Hoci mnohé z týchto výdavkov sú malé, ich objem je značný a účtovníctvo sa často prehliada. To vedie k množstvu problémov, od straty údajov a neefektívnych kontrol až po nedostatok transparentnosti vo finančných transakciách.

Väčšina správ o menších nákupoch sa ukladá v excelovských tabuľkách, čo neposkytuje potrebnú úroveň spoľahlivosti a transparentnosti, pretože ktokoľvek môže údaje sfalšovať, a to aj náhodným pochybením.

Technologický pokrok ponúka príležitosti na automatizáciu takýchto procesov. Vytvorenie online denníka menších nákupov môže výrazne zjednodušiť prácu študentskej rady tým, že poskytuje užívateľsky prívetivé rozhranie na vedenie záznamov, ukladanie informácií a generovanie správ.

Technológia blockchain, ktorá zvyšuje dôveru a transparentnosť vo finančných transakciách, je obzvlášť účinná pri riešení takýchto problémov. Decentralizovaná štruktúra zabezpečuje, že údaje nemožno meniť a všetky záznamy o nákupoch sú overiteľné. Týmto spôsobom pomáha udržiavať čestnosť a transparentnosť so študentmi pri rozdeľovaní finančných prostriedkov.

Hlavným cieľom tejto bakalárskej práce je vyvinúť prototyp decentralizovanej aplikácie (DApp) na vytváranie a vykonávanie mikrotendrov študentskou radou TUKE. Táto aplikácia zabezpečí automatizáciu a transparentnosť procesu obstarávania.

Systém umožní členom študentskej rady iniciovať požiadavky na obstarávanie, zhromažďovať ponuky od dodávateľov, vykonávať schvaľovanie kvóra a spracovávať platby pomocou inteligentných zmlúv na blockchaine Polygon. Všetky dokumenty a záznamy o transakciách budú uložené v decentralizovanom úložisku IPFS, čím sa zabezpečí ich dostupnosť pre každého študenta a ochrana pred úpravami.

Na dosiahnutie cieľa tejto práce budú implementované nasledujúce kroky:

1. Analýza finančných procesov. Študovať procesy obstarávania prostredníctvom tendrov, vrátane tých, ktoré vykonávajú študentské organizácie, a identifikovať kľúčové problémy, ktoré je potrebné riešiť.

2. Preskúmať existujúce riešenia. Porovnať výhody a nevýhody systémov tendrov, vrátane tých, ktoré nevyužívajú technológiu blockchain, a zhodnotiť ich vhodnosť pre študentské organizácie.

3. Analýza existujúcich technológií (blockchain, IPFS, DApp). Štúdium princípov blockchainu a inteligentných zmlúv, ako aj výber správnej platformy pre danú

prácu.

4. Vývoj a implementácia architektúry systému. Návrh architektúry systému, modelu databázy, API a UI/UX. Vývoj inteligentných zmlúv, backendových a frontendových aplikácií.

5. Testovanie. Testovanie funkčnosti, bezpečnosti a výkonu systému v praxi.

# 2 A little bit background

V tejto kapitole preskúmame existujúce systémy výberových konaní, aby sme pochopili základné koncepty, ktoré sú v nich obsiahnuté, a neskôr ich využili v našej práci. Taktiež sa oboznámime so základnými pojmami o blockchaine, konkrétne čo to je a ako funguje, bez hlbšieho ponárania sa do technickej časti.

## 2.1 Existujúce systémy tendrov

Existuje mnoho riešení určených pre účtovníctvo obstarávania a finančné riadenie. Väčšina z nich je však určená pre korporátny alebo vládny sektor a nie je vhodná pre potreby menších organizácií vrátane študentských rád. Analýzou a štúdiom týchto systémov je však možné identifikovať niektoré princípy, na ktoré sa možno odvolávať pri vývoji vlastného produktu.

Ako príklad si vezmime tri systémy verejných zákaziek v rôznych krajinách a skúsme ich porovnať.

- Prozorro [1]
- TenderNed (Holandsko)[2]
- Elektronický kontraktačný systém (Slovensko)[3]

Medzi ich výhody patrí vysoká úroveň transparentnosti a právnej ochrany, ako aj podávanie správ a audit, keďže tendre sa vykonávajú na vládnej úrovni. Hlavnou nevýhodou však je, že všetky tieto systémy tendrov sú určené pre veľké podniky a korporácie.

[1] https://prozorro.gov.ua/en

[2] https://www.tenderned.nl/cms/en/english/tenderned-dutch-governments-online--tendering-system

[3] https://www.eks.sk/

Tabuľka 2.1: Porovnanie elektronických verejných obstarávacích systémov

| Systém | Krajina | Opis | Výhody | Nevýhody |
|---|---|---|---|---|
| Prozorro | Ukrajina | Národný systém verejného obstarávania navrhnutý na boj proti korupcii | Transparentnosť, otvorené dáta, verejná kontrola | Komplexný pre malé nákupy, potreba školenia používateľov |
| EKS | Slovensko | Elektronický kontraktačný systém pre malé a stredné verejné obstarávania | Jednoduchosť, rýchlosť, transparentnosť | Neobľúbenosť medzi podnikateľmi, zastarané rozhranie |
| TenderNed | Holandsko | Štátna platforma pre všetky verejné tendre v krajine | Centralizovaný prístup, otvorené dáta, vysoká spoľahlivosť | Jazyková bariéra, zložitosť pre menšie organizácie |

Na základe uvedenej tabuľky2.1 je potrebné zohľadniť všetky pozitívne aspekty existujúcich tendrov (rýchlosť, transparentnosť, spoľahlivosť) a opraviť tie negatívne (vytvoriť jednoduché rozhranie postačujúce pre malé nákupy určené len pre študentskú radu).

## 2.2 Tender system evaluation

Náš systém výberového konania musí v sebe kombinovať všetky výhody korporátnych a štátnych systémov (bezpečnosť, pohodlnosť, transparentnosť), no zároveň odstraňovať ich nedostatky (nedostupnosť pre malé organizácie, centralizovanosť). Preto musí byť náš systém spoľahlivý a vzbudzovať dôveru používateľov. Nemenej dôležité je, aby bol transparentný a overiteľný všetkými používateľmi. Na tento účel využijeme technológiu blockchain, ktorá tento problém efektívne rieši. Vďaka nej si každý záujemca bude môcť prezrieť celý reťazec a skontrolovať celý proces obstarávania.

Nemenej podstatnou súčasťou je skutočnosť, že členovia študentskej rady budú musieť nejakým spôsobom dospieť k spoločnému rozhodnutiu — konkrétne, ktorá z ponúk je najvhodnejšia. V tom nám pomôže konsenzus, jednoduchšie povedané — hlasovanie. Blockchain podporuje možnosť hlasovania, a preto sa stane neoddeliteľnou súčasťou nášho systému. Podrobnejšie sa o tom bude hovoriť v nasledujúcej kapitole 3.

## 2.3 Blockchain technology

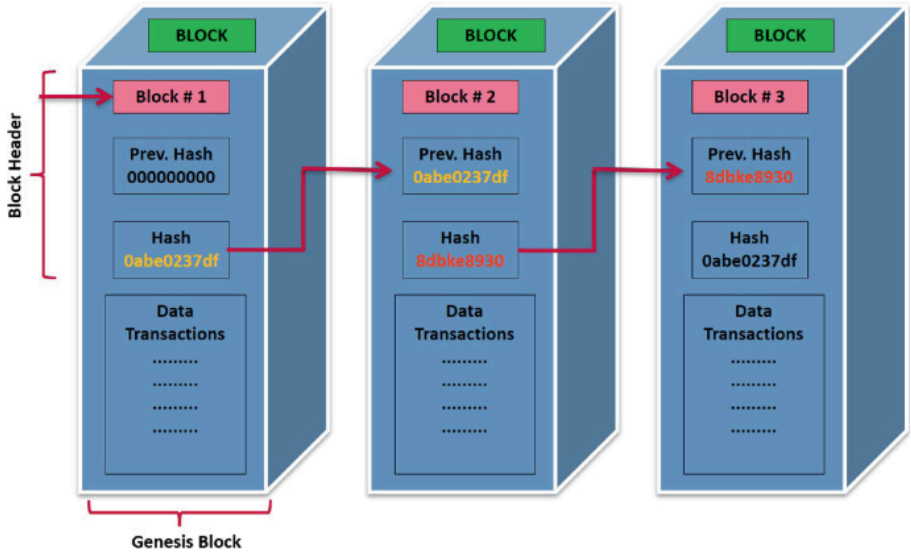
Blockchain je distribuovaná digitálna účtovná kniha, ktorá ukladá dáta akéhokoľvek typu. Blockchain dokáže zaznamenávať informácie o transakciách s kryptomenami, vlastníctve NFT alebo inteligentných zmluvách DeFi [**ardrive_blockchain_files**].

Zatiaľ čo akákoľvek tradičná databáza dokáže ukladať takéto informácie, blockchain je jedinečný v tom, že je úplne decentralizovaný. Namiesto toho, aby bol uložený na jednom mieste centralizovaným správcom – napríklad v tabuľke programu Excel alebo v bankovej databáze – viacero identických kópií blockchainovej databázy je uložených na viacerých počítačoch roztrúsených po sieti. Tieto jednotlivé počítače sa nazývajú uzly [**moonpay_blockchain_nodes**].

Vďaka tejto vlastnosti rieši používanie blockchainu problém dôvery, pretože žiadna osoba nemôže meniť žiadne dáta pre svoje vlastné účely. Bude jasné, do ktorej peňaženky[4] note boli peniaze prevedené, kedy, od koho atď.

### 2.3.1 General Working Sequence of Blockchain

Obrázok 2.1 zobrazuje obsah bloku v blockchainovej sieti, pričom konkrétne ukazuje hlavičku bloku a blok genesis.




Obrázok 2.1: Príklad architektúry blockchainového systému [**Rehmani2021**]

Nižšie rozoberieme všeobecnú pracovnú postupnosť blockchainu [**Rehmani2021**]:

1. Používateľ(i) blockchainu vytvorí(jú) účet(y).
2. Používateľ blockchainu vytvorí transakciu (Tx).
3. Transakcie sú podpísané.
4. Transakcia je vysielaná na overovacie uzly.
5. Transakcia je validovaná.
6. Transakcie sú zhromaždené v poole.
7. Uzol(y) ťažobného(ých) uzla(ov) zhromaždí(jú) viacero Tx a vytvorí blok.

[4] Crypto-wallet

8. Uzol(y) ťažobného(ých) uzla(ov) vykonáva(jú) ťažbu.
9. Vykoná sa validácia bloku.
10. Úspešný uzol ťažobného pridá blok do blockchainu.
11. Informácie o pridanom bloku sú vysielané do blockchainovej siete.
12. Uzol blockchainu pridá vysielaný blok do svojej lokálnej kópie blockchainu – potvrdenie bloku.
13. Blok sa stáva súčasťou globálneho blockchainu – blok je publikovaný.

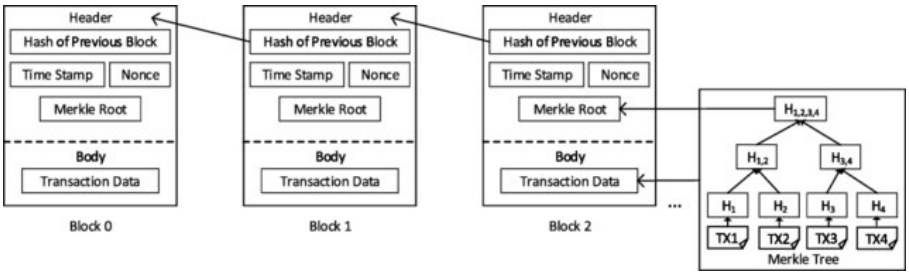
### 2.3.2 Transaction

Transakcie môžu tiež obsahovať súbor informácií. Používateľ môže vytvoriť jednu alebo viac transakcií. Vytvorenie týchto transakcií môže od používateľa blockchainu vyžadovať rôzne informácie a môžu sa líšiť v závislosti od implementácie blockchainu. Transakcia musí obsahovať všeobecné informácie, ako napríklad prenášané aktívum. Vyžadujú sa aj informácie o odosielateľovi a príjemcovi. Okrem toho musia byť zahrnuté informácie o vysporiadaní transakcie[**Rehmani2021**].

Po vytvorení transakcie je potrebné ju podpísať a následne overiť. Po podpísaní sa transakcie vysielajú do blockchainovej siete, kde ich overovacie uzly overia a pridajú do bloku. Ťažobné uzly potom overia bloky pomocou konsenzuálneho algoritmu a ak je všetko v poriadku, blok sa považuje za úspešne vyťažený. Keď sa blok považuje za úspešne vyťažený, jeho informácie už nemožno meniť. Akékoľvek zmeny je možné vykonať iba pridaním nových blokov[**ledger2025blockchainTransaction**].

### 2.3.3 Composition of a Block

Blok sa skladá z niekoľkých vecí. Obrázok zobrazuje komponenty bloku, ktoré popíšeme nižšie:




Obrázok 2.2: The structure of a Blockchain[**Liang2020**]

Blok sa skladá z hlavičky a tela, kde hlavička obsahuje hash predchádzajúceho bloku, časovú pečiatku, Nonce a Merkleho koreň. Merkleho koreň je hash koreňa Merkleho stromu, ktorý je uložený v tele bloku. Transakciu označujeme ako TX

a ako príklad na ilustráciu štruktúry Merkleho stromu berieme tretí blok, ktorý obsahuje iba štyri transakcie [**Liang2020**].

### 2.3.4 Blockchain Platforms

Tabuľka zobrazuje aplikáciu blockchainu a rôzne blockchainové systémy, ktoré takéto aplikácie podporujú.

Tabuľka 2.2: Hlavné oblasti použitia rôznych blockchainových systémov

| **Application of blockchain** | **Blockchain systems** |
|---|---|
| Asset Management and Tracking | Corda, BigchainDB, Multichain, VeChain, Hyperledger Fabric |
| Own assets in terms of Token | Stellar, IOTA, Tezos, Algorand, Cardano |
| Cryptocurrency | Bitcoin, Litecoin, Dogecoin, Monero, Dash |
| Smart Contract-based new Business Logic | Parity, Ethereum, Quorum, Solana, Avalanche, Binance Smart Chain, Polygon |
| Decentralized Storage | Filecoin, Arweave, Storj |
| Identity Management | Sovrin, uPort, Civic |
| Decentralized Finance (DeFi) | Ethereum, Avalanche, Solana, Terra |
| Interoperability / Cross-chain | Polkadot, Cosmos |
| Privacy-focused Transactions | Zcash, Monero, Secret Network |

Ako vidíme, existuje obrovské množstvo blockchainov používaných v rôznych oblastiach. Musíme si vybrať jeden, ktorý použijeme pre náš systém platieb. Prioritou je, aby tento blockchain podporoval inteligentné zmluvy[**hacken2025smartContractP**
Na tento účel sa pozrieme na ďalšiu, podrobnejšiu tabuľku 2.3 porovnávajúcu najpopulárnejšie blockchainy, ktoré podporujú inteligentné zmluvy.

Tabuľka 2.3: Comparative Overview of Blockchain Platforms (2025)

| **Criterion** | **Ethereum** | **Polygon** | **BSC** | **Solana** |
|---|---|---|---|---|
| Launch Year | 2015 | 2017 (as Matic) | 2020 | 2020 |
| Type | Layer 1 | Layer 2 / Sidechain | Layer 1 | Layer 1 |
| Consensus Mechanism | Proof of Stake (PoS) | Proof of Stake (PoS) | Proof of Staked Authority (PoSA) | Proof of Stake (PoS) |
| Real TPS | ~25 | ~75 | ~300 | ~720 |
| Average Fee (2025) | ~$0.1285 | ~$0.003 | ~$0.02 | ~$0.008 |
| EVM Compatibility | Yes (native) | Yes (full) | Yes (full) | No (custom VM) |
| Smart Contract Languages | Solidity, Vyper | Solidity, Vyper | Solidity | Rust, C, C++ |
| Native Token | ETH | MATIC (POL) | BNB | SOL |
| Block Time | ~12s | ~2s | ~0.75s | ~0.4s |
| Daily Active Wallets | ~530K | ~1.7M | ~4M | ~2M |
| Decentralization Level(Count of validators) | ~973.0K | ~100 | ~45 | ~800 |
| Number of dApps[**dappradar2025chainsRanking**] | 5.000+ | 2,500+ | 6.000+ | 700+ |
| Ecosystem Size | Largest | Large | Large | Growing |

*Source: Data compiled from* [**chainspect2025ethereumSolana**, **chainspect2025bnbPolygon**, **tokenterminal2025explorer**].

### Gas Fees

Poplatky za transakciu predstavujú jedno z kľúčových kritérií pri výbere vhodného blockchainu. V kontexte nášho systému je tento faktor obzvlášť dôležitý, keďže počas procesu vytvárania ponuky môže dôjsť k dvom alebo viacerým transakciám. Ethereum je síce v súčasnosti lacnejšie ako v minulosti, avšak jeho transakčné poplatky sú historicky nestabilné — v období vysokého zaťaženia siete môžu výrazne vzrásť. Napríklad v marci 2024 dosiahli poplatky až 25 USD za transakciu[**tokentermin** Takáto volatilita predstavuje pre náš systém neprijateľné riziko, keďže by mohla nastať situácia, v ktorej by transakčné náklady prevýšili samotnú hodnotu obstarávania. Z tohto dôvodu sa výber zužuje na siete Solana a Polygon, ktoré ponúkajú výrazne nižšie a stabilnejšie transakčné náklady.

### Speed and performance

Z hľadiska rýchlosti a výkonu sú si väčšina porovnávaných blockchainov pomerne podobné a spĺňajú požiadavky nášho systému. Výnimkou je opäť Ethereum, ktorého priepustnosť a rýchlosť spracovania transakcií sú pre naše účely nedostatočné. Ostatné posudzované siete v tomto kritériu obstoja.

#### Solidity support (EVM Compatibility)

Najpopulárnejším jazykom na písanie inteligentných zmlúv je Solidity. Je veľmi užívateľsky prívetivý a široký a má veľa knižníc, takže v ňom budeme písať všetok kód pre náš systém výberových konaní. Solana bohužiaľ nie je vhodná, pretože potrebujeme iný programovací jazyk, dokonca viac ako jeden, ako napríklad Rust a C++[**willson2025solanaPolygonEthereum**]. To má svoje výhody, ako je lepšia bezpečnosť a výkon, ale medzi jeho hlavné nevýhody patrí zložitejšia krivka učenia a menej príležitostí na použitie v ekosystéme Web 3.0.

#### Decentralization and Security

Ethereum má najviac validátorov[**tokenterminal2025ethereumValidators**], čo zabezpečuje najlepšiu decentralizáciu zo všetkých konkurentov na tomto zozname. Na druhom mieste je Solana, ktorá má výrazne menej validátorov, ale stále sa môže pochváliť veľmi vysokou úrovňou decentralizácie. Polygon a BSC sú na poslednom mieste, ale to neznamená, že sú menej decentralizované; jednoducho majú menej validátorov potvrdzujúcich transakcie v sieti v porovnaní s prvým z nich.

#### Winner

Na základe uvedených kritérií je Polygon[5] pre naše účely najvhodnejšou voľbou vďaka nízkym nákladom na transakcie a podpore inteligentných zmlúv a Solidity. Tabuľka ukazuje, že Polygon je široko používaný, s približne 1.7 miliónmi peňaženiek používaných denne, a veľmi veľký počet dApp si vybral Polygon pre vývoj. Ethereum, ktorý je dobre otestovaný a používa ho prevažná väčšina dApp, by mohol byť dobrou alternatívou, ale trpí vo veľmi dôležitej oblasti: transakčných poplatkoch.[6] Mohla by nastať situácia, keď by sa ponuka 20 dolárov mohla ukázať ako nerentabilná, pretože transakcie by stáli 15-20 dolárov.

### 2.3.5 Smart-contracts

Smart contracts sú digitálne zmluvy uložené v blockchaine, ktoré sa automaticky vykonajú, keď sú splnené vopred určené podmienky[**ibmSC**]. Fungujú ako elektronické dohody a v našom systéme budú zohrávať kľúčovú úlohu. Prostredníctvom inteligentných zmlúv budeme definovať jednotlivé roly používateľov, pričom každá rola bude mať prístup len ku konkrétnym funkciám systému, ktoré budú taktiež implementované pomocou inteligentných zmlúv.

[5] https://polygon.technology/

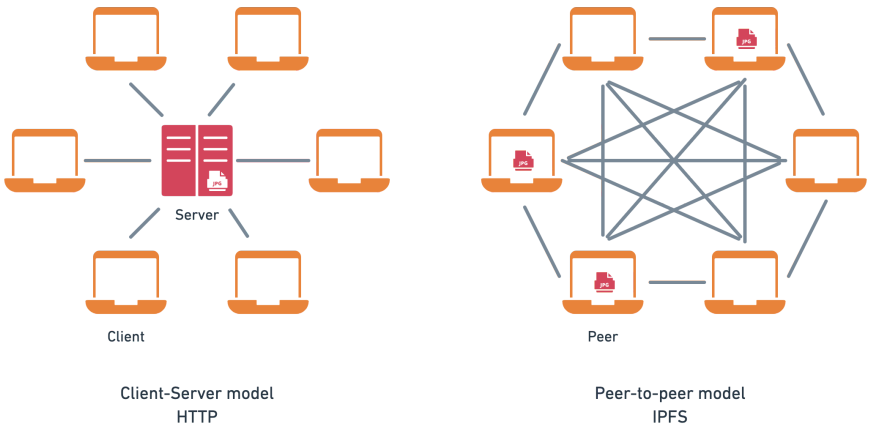
[6] Pri extrémnom zaťažení siete môžu byť poplatky výrazne vysoké.

### 2.3.6 Voting system

Blockchain sa v modernom svete čoraz častejšie využíva v oblasti hlasovania, keďže efektívne rieši problémy s integritou výsledkov a transparentnosťou celého procesu[**jafar2021blockchain**]. V našom systéme bude hlasovanie implementované za účelom výberu najvhodnejšej ponuky od dodávateľov. Keďže blockchain prirodzene podporuje anonymitu hlasujúcich, táto vlastnosť bude v našom riešení technicky dostupná, avšak vzhľadom na charakter prostredia – interné rozhodovanie študentskej rady – nie je jej využitie nevyhnutné.

## 2.4 IPFS

IPFS (Inter-Planetary File System) je distribuovaný súborový systém typu peer--to-peer, ktorý sa používa predovšetkým pre dáta, ktoré nemožno uložiť na blockchain. Na rozdiel od HTTP, štandardnejšieho protokolu na zdieľanie súborov, má IPFS špeciálne vlastnosti, vďaka ktorým je ideálny pre decentralizovaný dátový model, ktorý funguje v tandeme s blockchainmi[**pinata2025whatIsIpfs**].




Obrázok 2.3: Difference beetween IPFS and HTTP[**norman2026ipfsGateways**]

V tradičných webových aplikáciách sa na prenos a ukladanie dát používa protokol HTTP. V decentralizovaných systémoch, najmä v spojení s blockchain technológiami, sa však čoraz častejšie využíva IPFS, ktorý rieši viaceré obmedzenia centralizovaných riešení (Tab. 2.4).

Tabuľka 2.4: Porovnanie protokolov HTTP a IPFS

| **Kritérium** | **HTTP** | **IPFS** |
|---|---|---|
| Architektúra | Centralizovaná klient–server | Decentralizovaná peer-to-peer |
| Adresovanie obsahu | Podľa umiestnenia (URL, server) | Podľa obsahu (hash súboru – CID) |
| Odolnosť voči cenzúre | Nízka – server môže byť vypnutý alebo blokovaný | Vysoká – obsah je distribuovaný medzi uzlami |
| Integrita dát | Závisí od dôvery k serveru | Kryptograficky garantovaná hashom |
| Dostupnosť | Závislá od dostupnosti servera | Závislá od počtu dostupných uzlov |
| Škálovateľnosť | Obmedzená infraštruktúrou servera | Prirodzená vďaka distribúcii dát |
| Vhodnosť pre blockchain aplikácie | Obmedzená | Vysoká – často používaný pre ukladanie off-chain dát |

*Source: Data compiled from* [**pinata2025whatIsIpfs**, **IPFS**, **IBM_HTTP**].

### 2.4.1 Principles of IPFS operation

Základný princíp IPFS spočíva v tom, že súbory nie sú identifikované svojím umiestnením (URL), ale svojím obsahom. Keď je súbor pridaný do IPFS siete, systém vygeneruje unikátny kryptografický hash nazývaný Content Identifier (CID) [**cloudflareIPFS**]. Tento 24-znakový identifikátor je vytvorený pomocou SHA-256 algoritmu a slúži ako jednoznačný odkaz na obsah súboru [**howtogeekIPFS2022**].

IPFS používa Distributed Hash Table (DHT), konkrétne Kademlia DHT, na správu informácií o tom, ktoré uzly v sieti uchovávajú ktoré súbory [**IPFS**].

Keď používateľ požiada o súbor pomocou jeho CID, IPFS DHT vyhľadá uzly, ktoré tento súbor majú, a umožní priame stiahnutie od týchto uzlov. Veľké súbory sú rozdelené na menšie bloky (chunks), typicky 256 KB, a organizované do Merkle Directed Acyclic Graph (Merkle DAG) [**mediumIPFS2023**], čo umožňuje efektívnu deduplikáciu a paralelné sťahovanie z viacerých uzlov.

Dôležitým aspektom IPFS je koncept "pinningu" (pripevnenia). Bez pinningu sú súbory dostupné len pokiaľ aspoň jeden uzol v sieti ich aktívne uchováva [**lighthouse2026ipfsPinningGuide**]. IPFS pravidelne vykonáva garbage collection, ktorý maže súbory, ktoré nie sú pripnuté a neboli prístupované. Pre zabezpečenie trvalej dostupnosti dát sa používajú pinning services ako Pinata alebo Infura, ktoré garantujú, že súbory zostanú dostupné v sieti.

### 2.4.2 Why IPFS for documents?

Ukladanie dát priamo na blockchain je finančne neefektívne. Ako uvádza QuickNode, blockchainy ako Ethereum sú navrhnuté na ukladanie malých množstiev dát a väčšie súbory sa typicky ukladajú mimo reťazca pomocou IPFS [**quicknode2026ipfsEthereum**]. Na základe výpočtov nákladov na gas môže uloženie 1 MB dát stáť približne 1 USD na Polygon a výrazne viac na Ethereum.

Hybridný prístup kombinujúci blockchain a IPFS ponúka optimálne riešenie. Dokument sa uloží na IPFS a na blockchain sa zapíše len jeho CID hash, ktorý má fixnú veľkosť 32 bytov [**metana2026solidityIpfsIntegration**]. Tento prístup znižuje náklady o viac ako 99.99% pri zachovaní všetkých výhod blockchainovej technológie.

IPFS poskytuje pre dokumenty tendra niekoľko kľúčových výhod:

1. **Integrita dát:** CID je kryptografický hash obsahu súboru. Akákoľvek zmena v dokumente vedie k zmene CID, čo umožňuje automatickú detekciu manipulácie.

2. **Transparentnosť:** Každý môže stiahnuť dokument pomocou CID uloženého na blockchaine a overiť jeho autentickosť.

3. **Dostupnosť:** Dokumenty sú replikované naprieč viacerými uzlami v sieti, čo eliminuje single point of failure.

IPFS protokol samotný je bezplatný, avšak pre trvalú dostupnosť dát je bežne potrebné využiť pinningové služby, ktoré poskytujú replikáciu a záruku dostupnosti mimo vlastného uzla. Komerčné pinningové služby ako Filebase alebo Pinata ponúkajú platby začínajúce približne od $20 mesačne za stovky gigabajtov úložného priestoru [**IsIPFSStorageFreeFilebase**]. Pri plánovanom objeme údajov do niekoľkých stoviek megabajtov ročne by teda aj základný plán poskytoval dostatočný priestor bez vysokých nákladov. Naproti tomu priame ukladanie dát do blockchainu, najmä Ethereum, sa považuje za ekonomicky nevýhodné vzhľadom na vysoké poplatky za plyn pri ukladaní veľkých objemov dát [**WhyYourRollupShouldUseIPFSPinningS**

### 2.4.3 Alternative

Popri IPFS existujú aj iné decentralizované úložné riešenia, ktoré sme zvážili pre náš systém: Filecoin a Arweave.

Filecoin, taktiež vytvorený Protocol Labs, je blockchainová sieť postavená nad IPFS, ktorá pridáva ekonomickú vrstvu incentivizácie [**thetechtrends2026decentralizedStorag**
Zatiaľ čo IPFS definuje spôsob ukladania a prístupu k dátam, Filecoin vytvára

decentralizovaný trh, kde používatelia platia minerom za ukladanie dát pomocou FIL tokenov. Filecoin využíva kryptografické dôkazy Proof of Replication a Proof of Spacetime na zabezpečenie, že mineri skutočne uchovávajú dáta počas dohodnutého času [**aicerts2026decentralizedStorageSolutions**].

Nevýhodou Filecoinu je, že zmluvy o ukladaní dát majú zvyčajne obmedzenú životnosť a vyžadujú si pravidelné obnovovanie. Okrem toho je proces nastavenia zložitejší a vyžaduje si viac technických znalostí ako jednoduché používanie IPFS.

Arweave predstavuje zásadne odlišný prístup k decentralizovanému ukladaniu dát s dôrazom na trvalé ukladanie dát. Na rozdiel od IPFS a Filecoinu, ktoré si prenajímajú priestor na pevne stanovenú dobu, Arweave funguje na modeli „zaplať raz, ulož navždy“. Používatelia platia jednorazový poplatok v AR tokenoch a dáta sú trvalo uložené najmenej 200 rokov [**opensourceforu2026decentralisedStorage**].

Arweave je pre náš systém mikrotransakcií zbytočný. Dokumentácia k súťaži nevyžaduje trvalé uloženie, pretože zvyčajne musí byť prístupná len niekoľko rokov. Na základe nášho porovnania sme vybrali IPFS ako najvhodnejšie riešenie pre študentskú radu kvôli jeho optimálnej rovnováhe medzi cenou, výkonom a jednoduchou implementáciou.

Tabuľka 2.5: Porovnanie protokolov HTTP, IPFS, Filecoin a Arweave

| **Feature** | **IPFS** | **Filecoin** | **Arweave** |
|---|---|---|---|
| Storage Model | Peer-to-peer, no incentives | Incentivized, contract-based | Permanent storage, one--time fee |
| Best For | dApp hosting, NFT metadata | Long-term archival storage | Immutable, permanent data storage |
| Cost | Free (depends on pinning) | Market-based token economy | One-time upfront payment |
| Consensus Mechanism | None (protocol only) | Proof-of--Replication, PoSt | Blockweave |
| Example Projects | OpenSea, IPFS gateways | Starling Lab, Web3 archives | Mirror.xyz, ArDrive |

*Source: Data compiled from* [**aicerts2026decentralizedStorageSolutions**].

### 2.4.4 Integration with Blockain

IPFS a blockchain sú technológie, ktoré sú navzájom komplementárne. Zatiaľ čo blockchain poskytuje nemennosť, transparentnosť a konsenzus pre transakčné údaje a metadáta, IPFS ponúka efektívne riešenie pre ukladanie veľkých súborov a dokumentov.

V systéme mikrotenderov bude integrácia implementovaná nasledovne:

1. Najprv sa do IPFS nahrá dokumentácia k tendru (PDF súbor), kde jej bude priradené jedinečné CID.
2. Toto CID sa potom uloží do inteligentnej zmluvy na blockchaine Polygon spolu s ďalšími metadátami tendra (názov, rozpočet, termín atď.).
3. Inteligentná zmluva vygeneruje udalosť s CID a časovou pečiatkou, čím sa vytvorí overiteľný záznam o nahraní dokumentu.
4. Každý člen študentskej rady alebo študent bude môcť získať CID z blockchainu pomocou inteligentnej zmluvy a potom si stiahnuť dokument zo siete IPFS.
5. Integritu dokumentu je možné overiť porovnaním hashu nahraného súboru s CID uloženým v blockchaine.

Tento prístup zabezpečí transparentnosť, overiteľnosť a ochranu dokumentov pred neoprávnenou úpravou a zároveň minimalizuje naše náklady. Podobné riešenie už bolo úspešne implementované v rôznych aplikáciách, napríklad v decentralizovaných sociálnych sieťach [7].

Pre implementáciu použijeme službu pripnutia súborov Pinata[8], ktorá poskytuje jednoduché API na nahrávanie súborov a zaručuje ich dostupnosť v sieti IPFS.

[7] Kaichi je decentralizovaná sociálna sieť postavená na blockchaine Solana (https://kaichi.xyz/).

[8] How to Use Pinata: A Step-by-Step Guide for Beginners - https://pinata.cloud/blog/how-to-use-pinata-a-step-by-step-guide-for-beginners/

# 3 Requirements specification

Táto kapitola sa venuje analýze súčasného stavu procesov obstarávania v študentských radách, identifikácii ich hlavných nedostatkov a formulácii požiadaviek na navrhované riešenie. Cieľom je vytvoriť presné východisko pre návrh systému, ktorý bude tieto nedostatky adresovať prostredníctvom decentralizovaných technológií. Požiadavky vychádzajú z poznatkov získaných v analytickej časti práce (kapitola 2) a z praktických skúseností s fungovaním študentských organizácií.

## 3.1 Analysis of the current situation

Študentské rady vo vzdelávacích inštitúciách pravidelne uskutočňujú menšie nákupy pre potreby svojej činnosti – materiály na podujatia, kancelárske potreby, občerstvenie, propagačné predmety a podobne. Hoci jednotlivé sumy sú spravidla malé, ich kumulatívny objem počas akademického roka je značný a vyžaduje systematický prístup k evidencii a rozhodovaniu.

V súčasnosti tieto procesy typicky prebiehajú nasledovne:

- **Evidencia financií** je vedená v tabuľkových procesoroch (Microsoft Excel, Google Sheets). Záznamy o nákupoch, rozpočtoch a výdavkoch sú uložené lokálne alebo v cloudovom úložisku, pričom prístup k nim má obmedzený okruh osôb.
- **Komunikácia s dodávateľmi** prebieha neštruktúrovane prostredníctvom e-mailov, telefonátov alebo osobných kontaktov. Neexistuje jednotný systém na zber a porovnávanie ponúk.
- **Rozhodovanie o nákupoch** sa uskutočňuje na stretnutiach rady, kde sa diskutuje o prijatých ponukách a hlasuje sa o výbere dodávateľa. Proces je manuálny a závisí od fyzickej prítomnosti členov.
- **Dokumentácia** je fragmentovaná – ponuky, faktúry a zápisnice sa uchovávajú v rôznych formátoch a na rôznych miestach, bez centrálneho systému na ich správu a overenie.

Takýto spôsob fungovania je pre menšie organizácie bežný a pri nízkych objemoch transakcií môže byť postačujúci. S rastúcim počtom nákupov a zvyšujúcimi sa požiadavkami na transparentnosť sa však stáva zdrojom viacerých problémov, ktoré sú identifikované v nasledujúcej sekcii.

## 3.2 Problem identification

Na základe analýzy súčasného stavu bolo identifikovaných päť hlavných problémov, ktoré navrhované riešenie musí adresovať.

**P1: Nedostatok transparentnosti[distinto2022transparency].** Študenti, v mene ktorých rada nakladá s finančnými prostriedkami, spravidla nemajú prístup k informáciám o tom, aké nákupy boli uskutočnené, aké ponuky boli zvažované a na základe akých kritérií bol vybraný konkrétny dodávateľ. Neexistuje verejne dostupný a overiteľný záznam o priebehu obstarávacích procesov. Táto netransparentnosť môže viesť k nedôvere študentskej komunity voči rozhodnutiam rady.

**P2: Riziko manipulácie s dátami[eusprig2024spreadsheet].** Záznamy vedené v tabuľkových procesoroch môžu byť kedykoľvek upravené bez zanechania stopy o zmene. Neexistuje mechanizmus, ktorý by zabraňoval spätnej úprave údajov o ponukách, cenách alebo výsledkoch hlasovania. Aj keď úmyselná manipulácia nemusí byť bežná, samotná možnosť jej uskutočnenia oslabuje dôveryhodnosť celého procesu.

**P3: Neefektívne rozhodovanie.** Rozhodovanie o výbere dodávateľa vyžaduje zorganizovanie stretnutia rady, na ktorom sa členovia oboznámia s ponukami a hlasujú. Tento proces je časovo náročný a závisí od dostupnosti členov. Pri menších nákupoch je administratívna réžia spojená s rozhodovaním neprimeraná hodnote obstarávania.

**P4: Chýbajúca integrita dokumentov.** Ponuky dodávateľov, technické špecifikácie a ďalšie dokumenty existujú ako bežné súbory, ktoré je možné po ich odoslaní zmeniť alebo nahradiť. Neexistuje mechanizmus na overenie, že dokument predložený dodávateľom je totožný s dokumentom, na základe ktorého rada rozhodovala. Táto skutočnosť komplikuje prípadný spätný audit.

**P5: Absencia automatizovanej evidencie a vyhodnotenia[springer2025fraud].** Celý proces obstarávania – od zverejnenia výzvy, cez zbieranie ponúk, hlasovanie až po vyhlásenie výsledku – je spravovaný manuálne. Sledovanie stavov tendrov, počítanie hlasov a uchovávanie záznamov závisí od ľudského faktora, čo zvyšuje riziko administratívnych chýb a oneskorení.

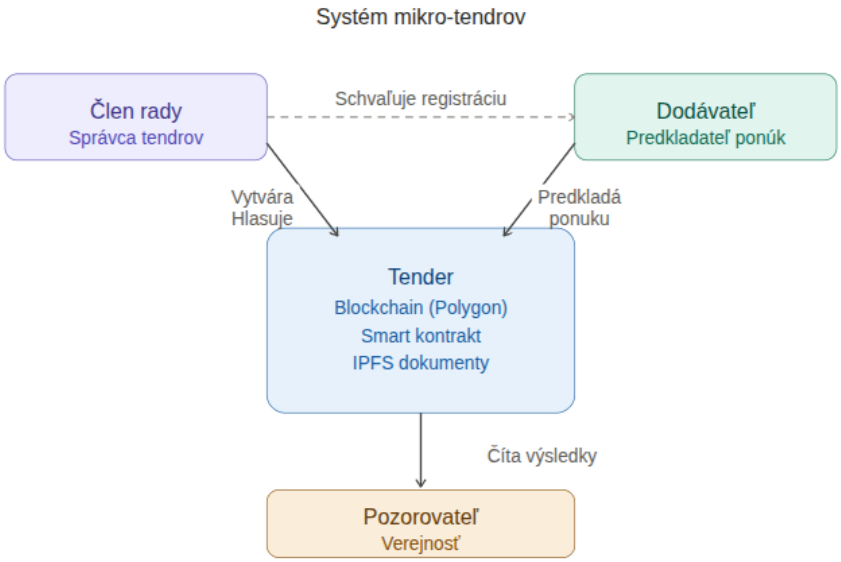
Uvedené problémy nie sú unikátne pre jednu konkrétnu organizáciu, ale sú

charakteristické pre širší okruh študentských rád a podobných samosprávnych orgánov, kde sa rozhoduje o nakladaní s kolektívnymi prostriedkami.

## 3.3 Functional requirements

Funkčné požiadavky definujú špecifické správanie a funkcie systému, ktoré musia byť implementované, aby sa dosiahli stanovené ciele. Požiadavky sú rozdelené podľa troch hlavných aktérov systému:

- **Člen študentskej rady** – osoba zodpovedná za vytváranie a správu tendrov, posudzovanie žiadostí dodávateľov, vyhodnocovanie ponúk a hlasovanie.
- **Dodávateľ** – fyzická alebo právnická osoba, ktorá sa registruje do systému a po schválení predkladá ponuky do zverejnených tendrov.
- **Verejnosť (pozorovateľ)** – akýkoľvek používateľ, ktorý má možnosť prezerať transparentné údaje o prebiehajúcich a ukončených tendroch bez potreby registrácie či pripojenia peňaženky.




Obrázok 3.1: Hlavní aktéri a jejich funkční požadavky v systému mikro--tendrov

### 3.3.1 Functional requirements for council members

**FR1: Vytvorenie tendra.** Člen rady musí mať možnosť vytvoriť nový tender zadaním nasledujúcich parametrov: názov, textový opis predmetu obstarávania, maximálny rozpočet, kategória obstarávania a voliteľne sprievodná dokumentácia vo formáte

PDF. Systém musí umožňovať priloženie dokumentácie nahratím súboru do decentralizovaného úložiska IPFS a uložením jeho obsahového identifikátora CID spolu s metadátami tendra na blockchain.

**FR2: Dvojkrokové publikovanie.** Vytvorenie tendra musí byť rozdelené na dva kroky: najprv vytvorenie konceptu (draft), kde je možné overiť a prípadne upraviť údaje vrátane CID dokumentácie, a následne publikovanie tendra so zadaním lehoty na predkladanie ponúk. Tento prístup znižuje riziko chybného zverejnenia.

**FR3: Správa životného cyklu tendra.** Člen rady, ktorý tender vytvoril, musí mať možnosť spustiť hlasovanie, finalizovať tender po ukončení hlasovania, označiť tender ako splnený a zrušiť tender v počiatočných fázach procesu.

**FR4: Hlasovanie o ponukách.** Každý člen rady musí mať možnosť odovzdať hlas práve jednej ponuke v rámci konkrétneho tendra. Hlasovanie musí prebiehať on-chain, čím sa zaručí transparentnosť a auditovateľnosť výsledkov. Systém musí zabrániť duplicitnému hlasovaniu tej istej osoby.

**FR5: Posudzovanie žiadostí dodávateľov.** Člen rady musí mať možnosť schváliť alebo zamietnuť žiadosť externého subjektu o registráciu ako dodávateľ. Schválením sa žiadateľovi udelí oprávnenie predkladať ponuky.

**FR6: Prezeranie ponúk a výsledkov.** Člen rady musí mať prístup k zoznamu všetkých predložených ponúk ku konkrétnemu tendru vrátane navrhovanej ceny, lehoty dodania a opisu, ako aj k výsledkom hlasovania po ukončení tendra.

### 3.3.2 Functional requirements for vendors

**FR7: Registrácia do systému.** Dodávateľ sa musí mať možnosť zaregistrovať prostredníctvom vyplnenia žiadosti obsahujúcej názov spoločnosti, kontaktné údaje a stručný opis činnosti. Žiadosť musí byť zaznamenaná na blockchaine a podliehať schváleniu členom študentskej rady.

**FR8: Prezeranie otvorených tendrov.** Registrovaný dodávateľ musí mať možnosť prezerať zoznam všetkých aktívnych tendrov, ich detaily vrátane rozpočtu, kategórie, lehoty uzávierky a prípadnej dokumentácie uloženej v IPFS.

**FR9: Predloženie ponuky.** Registrovaný dodávateľ musí mať možnosť predložiť ponuku ku konkrétnemu tendru zadaním cenovej ponuky, lehoty dodania a textového opisu riešenia. Systém musí validovať, že cena nepresahuje

maximálny rozpočet tendra, že tender je v stave prijímajúcom ponuky a že lehota uzávierky ešte neuplynula.

**FR10: Overenie ponuky na blockchaine.** Po podaní ponuky musí systém umožniť overenie jej zaznamenania na blockchaine prostredníctvom transakčného hashu a udalostí emitovaných smart kontraktom.

### 3.3.3 Functional requirements for the public

**FR11: Prezeranie tendrov bez registrácie.** Systém musí umožňovať akémukoľvek používateľovi prezerať zoznam všetkých tendrov, ich základné metadáta a aktuálny stav bez potreby pripojenia kryptografickej peňaženky alebo registrácie. Tým sa zabezpečuje transparentnosť procesu pre celú študentskú komunitu.

**FR12: Overenie výsledkov.** Verejnosť musí mať možnosť overiť výsledky hlasovania a víťaznú ponuku prostredníctvom verejne dostupných údajov na blockchaine, prípadne prostredníctvom blockchain exploreru.

**FR13: Prístup k dokumentácii.** Systém musí umožňovať prístup k dokumentácii tendra prostredníctvom IPFS, kde si používateľ dokument môže zobraziť alebo stiahnuť pomocou CID uloženého v kontrakte.

## 3.4 Non-functional requirements

Nefunkčné požiadavky definujú kvalitatívne atribúty systému, ktoré sú kritické pre jeho úspešné fungovanie. Sú rozdelené do piatich kategórií.

### 3.4.1 Bezpečnosť

**NFR1:** Všetky transakcie musia byť podpísané privátnym kľúčom používateľa prostredníctvom kryptografickej peňaženky, čím sa zabezpečuje autenticita každej operácie.

**NFR2:** Smart kontrakt musí byť otestovaný automatizovanými testami a manuálne skontrolovaný pred nasadením na sieť.

**NFR3:** Systém musí implementovať kontrolu prístupu založenú na rolách (RBAC), aby boli kritické operácie dostupné len oprávneným používateľom.

**NFR4:** Vstupné údaje musia byť validované priamo v smart kontrakte, aby sa predišlo ukladaniu neplatných alebo škodlivých dát na blockchain.

### 3.4.2 Výkonnosť

**NFR5:** Potvrdenie transakcie by malo byť dostupné do niekoľkých sekúnd vďaka použitiu siete Polygon, ktorá poskytuje výrazne kratšie časy bloku v porovnaní s hlavnou sieťou Ethereum.

**NFR6:** Transakčné poplatky (gas fees) by mali byť minimálne a pre bežné operácie by nemali presiahnuť zlomky centu, čo je dosiahnuteľné na testovacej sieti Polygon Amoy.

**NFR7:** Operácie čítania dát z kontraktu (view funkcie) musia byť dostupné bez transakčných poplatkov a s okamžitou odozvou.

### 3.4.3 Použiteľnosť

**NFR8:** Používateľské rozhranie musí byť intuitívne a prehľadné, aby systém mohli používať aj osoby bez predchádzajúcich skúseností s blockchain technológiou.

**NFR9:** Systém musí podporovať pripojenie prostredníctvom peňaženky MetaMask, ktorá je najrozšírenejšou peňaženkou pre interakciu s EVM-kompatibilnými sieťami.

**NFR10:** Rozhranie musí byť responzívne a použiteľné na zariadeniach s rôznymi veľkosťami obrazovky.

### 3.4.4 Spoľahlivosť a dostupnosť

**NFR11:** Kľúčové dáta (tendre, ponuky, hlasy) musia byť uložené na blockchaine, čím je garantovaná ich perzistencia a odolnosť voči výpadkom centrálnych serverov.

**NFR12:** Dokumenty uložené v IPFS musia byť dostupné prostredníctvom mechanizmu pinning, ktorý zabezpečuje ich dlhodobú perzistenciu v sieti.

**NFR13:** Údaje zaznamenané na blockchaine musia byť nemenné – po vytvorení záznamu nesmie byť možné ho spätne modifikovať ani vymazať.

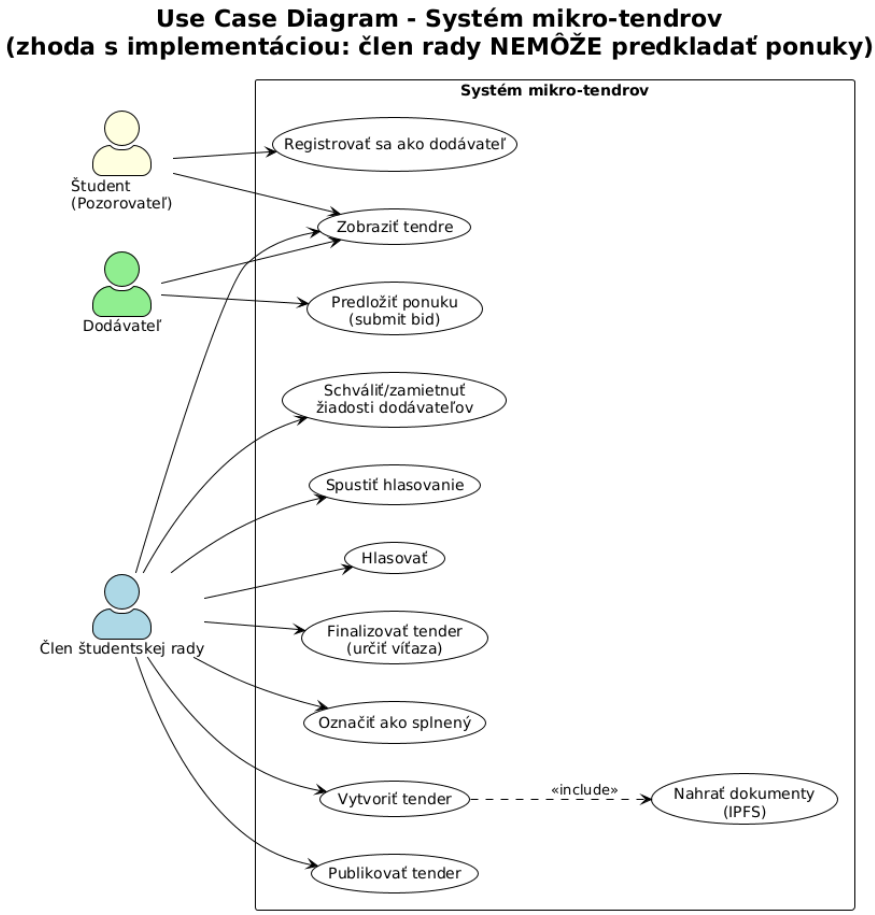
### 3.4.5 Transparentnosť

**NFR14:** Všetky kľúčové operácie musia byť overiteľné prostredníctvom blockchain exploreru (Polygonscan), kde si ktokoľvek môže overiť obsah transakcií a emitovaných udalostí.

**NFR15:** Systém musí v reálnom čase zobrazovať aktuálny stav každého tendra, aby bolo zrejmé, v akej fáze sa obstarávací proces nachádza.

## 3.5 Use cases

Diagram prípadov použitia (Use Case Diagram) znázorňuje interakcie medzi aktérmi a systémom. Hlavnými aktérmi sú člen študentskej rady, dodávateľ a verejnosť (pozorovateľ).




Obrázok 3.2: Diagram prípadov použitia pre systém mikro-tendrov

Z diagramu vyplývajú nasledujúce hlavné prípady použitia:

- **UC1: Vytvorenie a publikovanie tendra** – člen rady vytvorí koncept tendra, prípadne nahrá dokumentáciu do IPFS, a následne tender publikuje s definovanou lehotou.
- **UC2: Registrácia dodávateľa** – externý subjekt podá žiadosť o registráciu, ktorú člen rady schváli alebo zamietne.

- **UC3: Predloženie ponuky** – registrovaný dodávateľ podá ponuku ku konkrétnemu aktívnemu tendru.
- **UC4: Hlasovanie o ponukách** – členovia rady hlasujú za preferovanú ponuku v rámci hlasovacej fázy tendra.
- **UC5: Finalizácia a uzavretie tendra** – tvorca tendra ukončí hlasovanie, systém určí víťaznú ponuku a tender je administratívne uzavretý.
- **UC6: Prezeranie tendrov** – akýkoľvek používateľ vrátane verejnosti si môže prezerať zoznam tendrov, ich detaily, ponuky a výsledky hlasovania.

Jednotlivé prípady použitia pokrývajú celý životný cyklus tendra od jeho vytvorenia, cez registráciu dodávateľov a predkladanie ponúk, až po hlasovanie a finálne vyhodnotenie. Podrobný návrh procesov a interakcií je rozpracovaný v kapitole 4.

# 4 Návrh riešenia

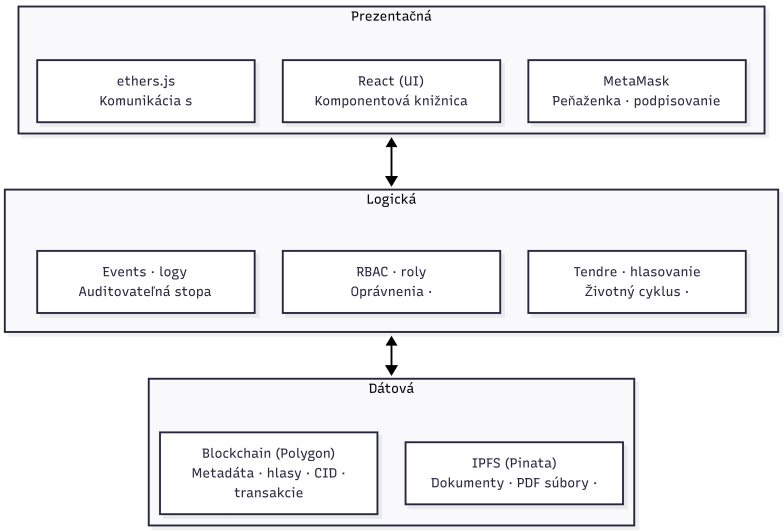
Predmetom tejto kapitoly je koncepčný návrh decentralizovanej aplikácie pre mikro--tendre študentských rád. Návrh vychádza z požiadaviek formulovaných v kapitole 3 a z výsledkov analýzy existujúcich riešení uvedených v kapitole 2. Cieľom nie je navrhnúť univerzálny systém verejného obstarávania, ale vytvoriť funkčný prototyp, ktorý zodpovedá špecifickému kontextu malých nákupov študentských organizácií. Navrhované riešenie preto musí byť technologicky primerané rozsahu bakalárskej práce a zároveň adresovať hlavné identifikované problémy: nedostatok transparentnosti, manuálnu administratívu, slabú auditovateľnosť a obmedzenú integritu dokumentácie.

Kapitola najprv opisuje celkovú architektúru systému a zdôvodňuje kľúčové architektonické rozhodnutia. Následne sa venuje návrhu smart kontraktu, jeho dátovým štruktúram, rolám a stavovému modelu. Ďalšie sekcie pokrývajú životný cyklus tendra, integráciu s IPFS, návrh používateľského rozhrania a nakoniec bezpečnostné a transparenčné aspekty riešenia.

## 4.1 Celková architektúra systému

### 4.1.1 Prehľad architektúry

Navrhovaný systém je založený na trojvrstvovej architektúre, ktorá je v prostredí Web3 prirodzeným a praktickým riešením. V porovnaní s tradičnými webovými aplikáciami, kde sa medzi klientom a databázou typicky nachádza aplikačný server, tu rozhodujúca časť logiky prebieha priamo v smart kontrakte nasadenom na blockchaine. Systém preto neobsahuje samostatný backendový server ani relačnú databázu.




Obrázok 4.1: Architektúra systému MicroTender: prezentačná vrstva (React), logická vrstva (smart kontrakt) a dátová vrstva (blockchain + IPFS).

Architektúru možno rozdeliť na tri základné vrstvy:

1. **Prezentačná vrstva** je tvorená webovou aplikáciou implementovanou v prostredí React. Používateľ komunikuje so systémom prostredníctvom grafického rozhrania v prehliadači, prepája sa s peňaženkou MetaMask a odosiela transakcie do blockchainovej siete. Súčasťou prezentačnej vrstvy je aj celá logika pre zobrazovanie stavov tendrov, formulárov, detailov ponúk, správu používateľských rolí a nahrávanie dokumentov do IPFS.

2. **Logická vrstva** je reprezentovaná jedným smart kontraktom `MicroTender.sol`, nasadeným na testovacej sieti Polygon Amoy. Kontrakt implementuje pravidlá správy tendrov, registráciu dodávateľov, ponuky, hlasovanie a kontrolu oprávnení. Táto vrstva zastupuje funkciu, ktorú by v klasickej webovej architektúre vykonával aplikačný server. Na rozdiel od tradičného backendu je však logika smart kontraktu po nasadení nemenná, verejne overiteľná a vykonávaná deterministicky všetkými uzlami siete.

3. **Dátová vrstva** je rozdelená medzi blockchain a IPFS. Na blockchaine sú uložené kľúčové metadáta potrebné pre auditovateľnosť a vynucovanie pravidiel: identifikátory tendrov, adresy tvorcov a dodávateľov, ceny ponúk, hlasy a stavové prechody. Väčšie dokumenty a súbory sú ukladané off-chain v sieti IPFS, pričom na blockchaine sa eviduje iba ich obsahový identifikátor CID. Takéto rozdelenie znižuje náklady na gas a zároveň zachováva možnosť overenia integrity dokumentov.

Takto navrhnutá architektúra znižuje počet samostatných komponentov, ktoré je potrebné vyvíjať a udržiavať. Systém neobsahuje žiadny middleware, message broker ani cachovaciu vrstvu. Z pohľadu bakalárskej práce ide o vedomý kompromis medzi technickou jednoduchosťou a dostatočnou funkcionalitou prototypu.

### 4.1.2 Technologický stack

Pri návrhu riešenia bol zvolený technologický stack, ktorý zodpovedá požiadavke na jednoduchú, ale funkčnú implementáciu prototypu. Voľba jednotlivých technológií bola motivovaná najmä ich zrelosťou, kvalitou dokumentácie, kompatibilitou s decentralizovany prostredím a dostupnosťou pre vývojára pracujúceho na bakalárskom projekte.

- **React** (verzia 19)[**reactDocs**] slúži na implementáciu prezentačnej vrstvy. Jeho komponentový model umožňuje rozdeliť rozhranie na znovupoužiteľné časti, čo zjednodušuje údržbu kódu. Pre správu stavu aplikácie sa využívajú zabudované React hooks (`useState`, `useEffect`, `useRef`), bez potreby externého state management riešenia.

- **ethers.js** (verzia 6)[**ethersDocs**] zabezpečuje komunikáciu medzi frontendom a blockchainom. Knižnica poskytuje rozhranie pre čítanie údajov z kontraktu prostredníctvom view funkcií, podpisovanie a odosielanie transakcií cez pripojenú peňaženku a spracovanie udalostí emitovaných kontraktom. Oproti alternatíve web3.js ponúka ethers.js menšiu veľkosť balíka a modernejšie API.

- **MetaMask**[**metamaskDocs**] slúži ako kryptografická peňaženka a nástroj na autorizáciu operácií. Každá transakcia vyžaduje explicitný súhlas používateľa v rozhraní MetaMask, čím sa zabezpečuje, že žiadna operácia nemôže prebehnúť bez vedomia vlastníka adresy.

- **Solidity**[**soliditiyDocs**] (verzia 0.8.19) je použitá na implementáciu smart kontraktu. Táto verzia bola zvolená pre vstavanú ochranu proti pretečeniu celočíselných typov (overflow/underflow), ktorú poskytuje od verzie 0.8.0, čím odpadá potreba externej knižnice SafeMath.

- **Hardhat**[**hardhatDocs**] poskytuje vývojové prostredie pre kompiláciu, testovanie a nasadzovanie kontraktu. Jeho výhodou je integrované lokálne prostredie Hardhat Network, podpora pre automatizované testy v JavaScripte a jednoduchá konfigurácia pre viaceré siete.

- **IPFS**[**IPFS**] a služba **Pinata**[**pinataDocs**] sú využité na ukladanie dokumentácie spojenej s tendrami. Pinata poskytuje REST API pre nahrávanie súborov a garantuje ich dostupnosť prostredníctvom mechanizmu pinning, čím sa predchádza automatickému odstráneniu obsahu z IPFS siete.
- **Tailwind CSS**[**tailwindDocs**] je využitý na štýlovanie komponentov rozhrania. Utility-first prístup umožňuje rýchlu iteráciu dizajnu priamo v JSX kóde bez písania samostatných CSS súborov.
- **Polygon Amoy**[**polygonAmoyDocs**] je testovacia sieť Polygon blockchainu, na ktorej je možné overovať správanie aplikácie bez použitia produkčných prostriedkov. V porovnaní s hlavnou sieťou Ethereum ponúka Polygon výrazne nižšie transakčné poplatky a rýchlejšie potvrdenie blokov, čo ho robí vhodným pre prototypovanie.

Uvedený stack pokrýva všetky vrstvy navrhovanej architektúry a nevyžaduje prevádzku žiadnej serverovej infraštruktúry okrem samotného blockchainu.

### 4.1.3 Komunikácia medzi komponentmi

Komunikácia v systéme prebieha dvoma hlavnými kanálmi, ktoré zodpovedajú dvom typom operácií: zápisu na blockchain a práci s dokumentmi.

#### Komunikácia s blockchainom

Pri operáciách, ktoré menia stav systému – napríklad pri vytvorení tendra, podaní ponuky alebo hlasovaní – frontend komunikuje cez MetaMask a knižnicu ethers.js priamo so smart kontraktom. Každá takáto operácia vyžaduje potvrdenie používateľa v peňaženke, podpísanie transakcie privátnym kľúčom a následné zapísanie do blockchainu. Výsledok operácie je k dispozícii po zahrnutí transakcie do bloku, čo v sieti Polygon Amoy trvá spravidla niekoľko sekúnd. Smart kontrakt pri každej dôležitej operácii emituje udalosť (event), ktorú frontend môže spracovať na potvrdenie úspešnosti transakcie.

#### Čítanie dát z kontraktu

Prebieha prostredníctvom view funkcií, ktoré nevyžadujú transakciu ani potvrdenie v peňaženke. Frontend volá tieto funkcie cez ethers.js a výsledky sú vrátené priamo z lokálneho uzla siete. Medzi najdôležitejšie view funkcie patria `getTender()`, `getTenderBids()`, `getVoteCount()`, `getUserRole()` a `isRegisteredVendor()`.

**Komunikácia s IPFS.**

Pri práci s dokumentmi frontend komunikuje priamo so službou Pinata prostredníctvom REST API. Súbor je nahraný cez HTTP POST požiadavku, Pinata vráti identifikátor obsahu CID a frontend tento CID následne odovzdá smart kontraktu pri vytváraní tendra. Prístup k uloženým dokumentom prebieha cez verejne dostupné IPFS gateway, kde sa CID transformuje na URL adresu.

**Režim len na čítanie.**

Dôležitým aspektom návrhu je podpora pre nepripojených používateľov. Ak používateľ nemá pripojenú peňaženku MetaMask, aplikácia inicializuje read-only RPC provider a naďalej umožňuje prezeranie verejných údajov o tendroch. Používateľ v tomto režime nemôže vykonávať transakcie, ale má prístup k zoznamom tendrov, ich detailom a ponukám. Táto vlastnosť podporuje transparentnosť systému, pretože sprístupňuje informácie aj pozorovateľom, ktorí nechcú alebo nepotrebujú interagovať s blockchainom.

### 4.1.4 Architektonické rozhodnutia

V procese návrhu boli prijaté viaceré architektonické rozhodnutia, ktoré zásadne ovplyvňujú výslednú podobu systému. Uvedené rozhodnutia sú výsledkom posúdenia technických možností, rozsahu bakalárskej práce a reálnych potrieb cieľovej skupiny.

**Absencia backendu.**

Systém neobsahuje žiadny serverový backend. Všetka biznis logika je implementovaná v smart kontrakte a frontend komunikuje s blockchainom priamo. Tento prístup bol zvolený z viacerých dôvodov: znižuje celkovú komplexitu projektu, eliminuje potrebu prevádzky a zabezpečenia serverovej infraštruktúry a zachováva princíp priameho, overiteľného prístupu k dátam bez sprostredkovateľa. Pre rozsah prototypu je tento prístup dostatočný; pri prípadnom nasadení do produkcie by bolo možné doplniť indexačnú vrstvu pre optimalizáciu čítania dát.

**Absencia relačnej databázy.**

Blockchain slúži ako primárny zdroj pravdy pre všetky dáta v systéme. Aktuálny rozsah projektu nevyžaduje off-chain cache ani fulltextové vyhľadávanie, preto nebola zavedená žiadna databáza. Toto rozhodnutie zjednodušuje deployment a eliminuje potrebu synchronizácie medzi blockchainom a externým úložiskom.

**Jeden smart kontrakt namiesto viacerých.**

Celá logika systému je implementovaná v jednom kontrakte. Táto voľba je podrobnejšie zdôvodnená v sekcii 4.2.1.

## 4.2 Návrh smart kontraktu MicroTender

Smart kontrakt je rozhodujúcou súčasťou navrhovaného systému. Definuje pravidlá správy tendrov, riadenie prístupu, životný cyklus obstarávania, logiku hlasovania aj evidenciu ponúk. Všetky tieto funkcionality sú sústredené v jednom kontrakte `MicroTender.sol`, ktorý je napísaný v jazyku Solidity verzie 0.8.19 a nasadený na sieti Polygon Amoy.

### 4.2.1 Monolitická architektúra kontraktu

Celé riešenie je implementované v jednom smart kontrakte. Alternatívny prístup by spočíval v použití viacerých kontraktov alebo factory patternu, kde by každý tender bol reprezentovaný samostatnou inštanciou kontraktu. Monolitický prístup bol uprednostnený z nasledujúcich dôvodov:

- **Nižšia zložitosť.** Všetka podstatná logika sa nachádza na jednom mieste, čo uľahčuje porozumenie systému, jeho testovanie a prípadné úpravy.
- **Nižšie náklady na gas.** Pri factory patterne by každé vytvorenie tendra vyžadovalo nasadenie nového kontraktu, čo je výrazne nákladnejšia operácia než zápis štruktúry do existujúceho mapovania.
- **Jednoduchšia správa stavu.** Všetky tendre, ponuky a hlasovanie sú spravované centralizovane v jednom kontrakte, čo eliminuje potrebu koordinácie medzi viacerými kontraktmi a zjednodušuje dotazovanie na globálne štatistiky.
- **Primeranosť rozsahu.** Pre prototyp bakalárskej práce zameraný na mikro--tendre s obmedzeným počtom súčasných tendrov je monolitický kontrakt technicky postačujúci.

Monolitický kontrakt zároveň lepšie zodpovedá charakteru mikro-tendrov. Systém pracuje s menšími obstarávaniami, pri ktorých je dôležitá jednoduchosť, prehľadnosť a nízka administratívna réžia. Z tohto dôvodu nebolo potrebné implementovať zložitejšie kontraktové väzby, escrow mechanizmy alebo samostatné spracovanie finančných tokov.

### 4.2.2 Dátové štruktúry

Systém pracuje s troma hlavnými entitami, ktoré modelujú proces obstarávania. Prvou je **tender** — základná jednotka systému, ktorá uchováva názov, opis, maximálny rozpočet, kategóriu, stav a voliteľný odkaz na dokumentáciu uloženú v sieti IPFS.

Druhou entitou je **ponuka** dodávateľa, ktorá obsahuje navrhovanú cenu, lehotu dodania a textový opis riešenia.

Treťou entitou je **žiadosť o registráciu**, prostredníctvom ktorej externý subjekt žiada o oprávnenie vstúpiť do procesu obstarávania.

Všetky entity sú uložené priamo na blockchaine, čím sa zabezpečuje ich nemennosť a verejná overiteľnosť [**Rehmani2021**]. Väčšie súbory — napríklad sprievodná dokumentácia tendra — nie sú ukladané priamo do kontraktu, ale do decentralizovaného úložiska IPFS [**pinata2025whatIsIpfs**]. Na blockchaine sa eviduje iba kryptografický identifikátor obsahu (CID), čo výrazne znižuje transakčné náklady pri zachovaní možnosti overiť integritu dokumentu.

### 4.2.3 Roly a oprávnenia

Systém rozlišuje štyri typy aktérov s rôznymi úrovňami prístupu. Princíp riadenia prístupu na základe rolí (RBAC) je v kontexte smart kontraktov bežne používaným prístupom na vynucovanie oprávnení [**jafar2021blockchain**].

**Administrátor** je vlastník kontraktu — spravidla predseda alebo správca rady. Má najvyššie oprávnenia: môže prideľovať a odoberať roly ostatným používateľom. Táto rola sa priraďuje automaticky pri nasadení systému.

**Člen rady** je používateľ, ktorému administrátor explicitne udelil oprávnenie. Členovia rady môžu vytvárať a publikovať tendre, hlasovať o ponukách a rozhodovať o žiadostiach dodávateľov.

**Dodávateľ** je externý subjekt, ktorý prešiel registračným procesom a bol schválený členom rady. Po schválení môže predkladať ponuky k aktívnym tendrom. Tento prístup umožňuje rade kontrolovať, kto sa obstarávania zúčastňuje.

**Pozorovateľ** nemá v systéme žiadnu priradenú rolu. Môže prezerať verejné informácie o tendroch a ich výsledkoch bez potreby registrácie či pripojenia peňaženky, čo podporuje transparentnosť procesu voči širšej študentskej komunite.

Tabuľka 4.1: Porovnanie rolí v procese obstarávania a v smart kontrakte

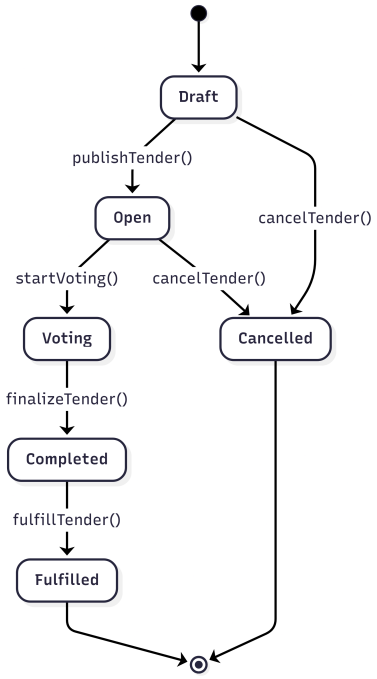
| **Rola** | **Úloha v procese** | **Reprezentácia v kontrakte** | **Kontrola prístupu** |
|---|---|---|---|
| Administrátor | Správa systému, prideľovanie členov rady | `UserRole.Admin` (hodnota `1`)[1] | Modifikátor `onlyOwner` |
| Člen rady | Vytvára tendre, hlasuje, schvaľuje dodávateľov | `UserRole.Member` (hodnota `0`)[1] | Modifikátor `onlyMember` |
| Dodávateľ | Registruje sa, predkladá ponuky k aktívnym tendrom | Nie je súčasťou `UserRole`[2]; evidovaný samostatne | Podmienka `registeredVendors[msg.sender]` |
| Pozorovateľ | Prezerá verejné údaje o tendroch a výsledkoch | Žiadna explicitná rola v kontrakte | Prístup len cez `view` funkcie |

[1] Roly `Admin` a `Member` sú definované ako enumerácia `UserRole` v Solidity: `Member = 0`, `Admin = 1`. Modifikátor `onlyMember` overuje podmienku `userRoles[msg.sender] >= UserRole.Member`, čím automaticky zahŕňa aj administrátorov.

[2] Dodávateľ nie je súčasťou enumerácie `UserRole`. Jeho oprávnenie je výsledkom schváleného registračného procesu a je evidované samostatne v mapovaní `registeredVendors`.

### 4.2.4 Stavový model tendra

Tender prechádza počas svojho životného cyklu šiestimi stavmi. Tento model zaručuje, že proces obstarávania prebieha v predvídateľnom poradí a žiadna fáza nemôže byť preskočená ani sa vrátiť späť [**ibmSC**].




Obrázok 4.2: Stavový model tendra, ktorý zaručuje správne poradie fáz procesu obstarávania.

Tender vzniká ako **koncept** (Draft), v ktorom ho tvorca môže upravovať a priložiť dokumentáciu. Po skontrolovaní údajov ho tvorca **zverejní** (Open) — od tohto momentu môžu registrovaní dodávatelia predkladať ponuky. Po uplynutí lehoty tvorca spustí **hlasovanie** (Voting), počas ktorého členovia rady vyberú najvhodnejšiu ponuku. Po ukončení hlasovania systém **vyhodnotí** výsledky (Completed) a určí víťaznú ponuku. Tvorca následne môže tender označiť ako **splnený** (Fulfilled). V počiatočných fázach je možné tender **zrušiť** (Cancelled).

Každý prechod medzi stavmi je podmienený splnením konkrétnych pravidiel, ktoré sú vynucované priamo v kóde kontraktu — nie organizačne. Tým sa eliminuje možnosť obísť niektorú z fáz procesu.

## 4.3 Životný cyklus tendra a proces obstarávania

Na úrovni používateľského procesu možno návrh systému rozdeliť do troch hlavných etáp: vytvorenie a publikovanie tendra, registrácia dodávateľa a podanie ponuky, hlasovanie a ukončenie tendra. Každá etapa zahŕňa interakciu medzi jedným alebo viacerými aktérmi a smart kontraktom.

### 4.3.1 Etapa 1: Vytvorenie a publikovanie tendra

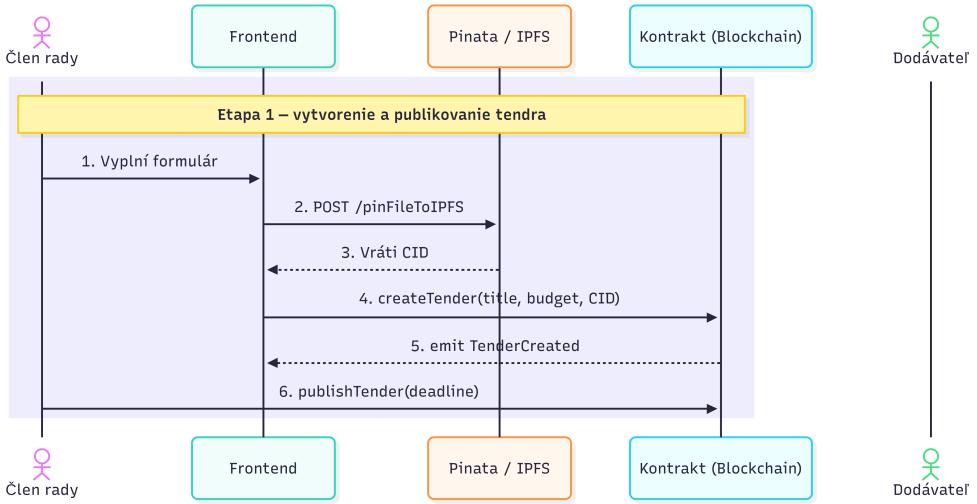
Proces začína členom rady, ktorý identifikuje potrebu obstarávania a pripraví základné údaje o tendri (obr. 4.3). Vo webovej aplikácii vyplní formulár obsahujúci názov, textový opis, maximálny rozpočet, kategóriu a voliteľne priloží dokumentáciu.

Ak je k tendru priložená dokumentácia, používateľ ju najprv nahrá do IPFS prostredníctvom služby Pinata. Frontend zabezpečí validáciu typu a veľkosti súboru (maximálne 10 MB, podporované formáty PDF, DOC a DOCX). Po úspešnom nahratí frontend obdrží CID, ktorý sa uloží do formulára a neskôr odovzdá kontraktu.

V návrhu aplikácie je tento proces riešený dvojkrokovo. Najprv sa tender vytvorí ako koncept pomocou funkcie `createTender()`, čím sa uloží v stave `Draft`. Táto funkcia prijíma päť parametrov: názov, opis, rozpočet v wei, kategóriu a CID dokumentácie. Kontrakt validuje, že názov nie je prázdny a nepresahuje `MAX_STRING_LENGTH` znakov, rozpočet je kladný a kategória je zadaná. Po úspešnom vytvorení sa emituje udalosť `TenderCreated` s identifikátorom tendra, adresou tvorcu a názvom.

V druhom kroku tvorca tender publikuje pomocou `publishTender()`, pri ktorom zadá lehotu na predkladanie ponúk v dňoch. Stav sa zmení na `Open` a emituje sa udalosť `TenderPublished`. Takéto rozdelenie umožňuje tvorcovi ešte pred publikovaním skontrolovať správnosť údajov, prípadne aktualizovať

CID dokumentácie prostredníctvom funkcie `updateTenderIPFSCID()`, ktorá je dostupná výlučne v stave `Draft`.




Obrázok 4.3: Vytvorenie a publikovanie tendra.

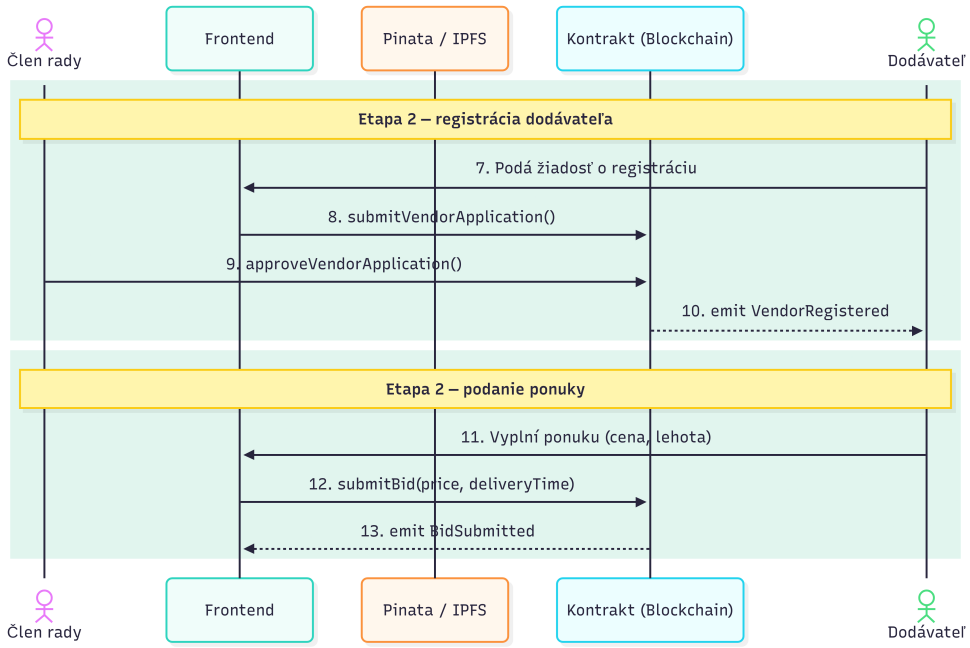
### 4.3.2 Etapa 2: Registrácia dodávateľa a podanie ponuky

Dodávateľ sa do systému nezapája automaticky pripojením peňaženky, ale musí najprv podať žiadosť o registráciu. Táto kontrola je v návrhu dôležitá, pretože umožňuje študentskej rade overovať, kto je oprávnený vstupovať do procesu obstarávania.

Dodávateľ vyplní formulár s názvom spoločnosti, kontaktnými údajmi a opisom činnosti, a odošle žiadosť prostredníctvom funkcie `submitVendorApplication()`. Kontrakt pritom overuje, že žiadateľ ešte nie je registrovaným dodávateľom a nemá rozpracovanú žiadosť. Po odoslaní sa emituje udalosť `VendorApplicationSubmitted`.

Člen rady následne žiadosť posudzuje a rozhodne o jej schválení alebo zamietnutí. Schválením cez `approveVendorApplication()` sa adresa žiadateľa zapíše do mapovania `registeredVendors` a emitujú sa udalosti `VendorApplicationApproved` a `VendorRegistered`. Zamietnutím cez `rejectVendorApplication()` sa stav žiadosti zmení na `Rejected` a žiadateľovi sa umožní podať novú žiadosť, pretože sa resetuje jeho záznam v mapovaní `vendorApplicationByAddress`.

Po schválení registrácie môže dodávateľ podávať ponuky k aktívnym tendrom pomocou funkcie `submitBid()`. Pri podávaní ponuky zadáva cenu v wei, lehotu dodania v dňoch a textový opis. Kontrakt validuje, že tender je v stave `Open`, termín uzávierky ešte neuplynul, cena je kladná a nepresahuje maximálny rozpočet tendra, lehota dodania je kladná a volajúci je registrovaný dodávateľ. Po úspešnom podaní sa emituje udalosť `BidSubmitted`.




Obrázok 4.4: Registrácia dodávateľa a podanie ponuky.

V aktuálnom prototype sa dokumentácia viaže primárne na samotný tender; samostatné pole CID pre každú ponuku nie je v štruktúre `Bid` implementované. Toto je vedomé zjednodušenie, ktoré by bolo možné rozšíriť v budúcich verziách.

### 4.3.3 Etapa 3: Hlasovanie a ukončenie tendra

Po skončení fázy prijímania ponúk môže tvorca tendra spustiť hlasovanie pomocou funkcie `startVoting()`. Podmienkou spustenia je, že tender je v stave `Open` a obsahuje aspoň jednu podanú ponuku. Tvorca zároveň zadáva počet dní na hlasovanie, pričom hodnota musí byť v rozsahu 1 až 14 dní podľa konštánt kontraktu. Tým sa tender presunie do stavu `Voting` a nastaví sa `votingDeadline`.

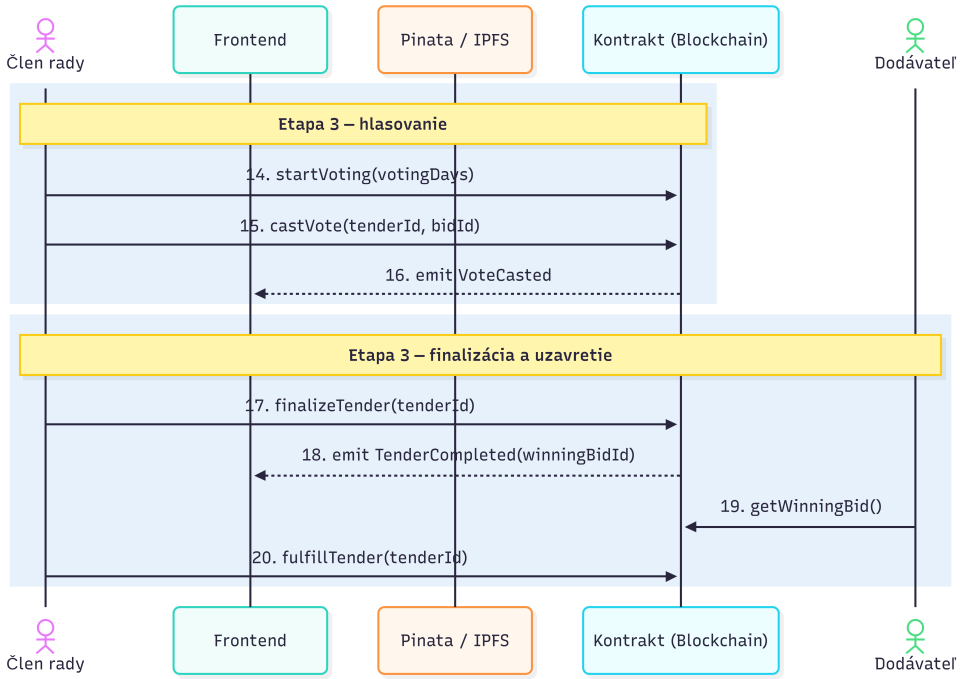
V hlasovacej fáze majú členovia rady prístup k zoznamu predložených ponúk. Každý oprávnený člen môže odovzdať hlas práve jednej ponuke prostredníctvom funkcie `castVote()`. Kontrakt kontroluje, či tender je v stave `Voting`, hlasovacie obdobie ešte neuplynulo, volajúci má rolu člena rady, dosiaľ nehlasoval v tomto tendri a zvolená ponuka skutočne existuje v poli `tenderBids`. Po úspešnom hlasovaní sa emituje udalosť `VoteCasted`.

Overenie existencie ponuky prebieha iteráciou cez pole ponúk tendra s optimalizovaným cyklom, kde sa inkrementácia indexu vykonáva v bloku `unchecked` pre zníženie spotreby gas.

Po uplynutí hlasovacej lehoty sa tender ukončí funkciou `finalizeTender()`. Kontrakt v tomto kroku iteruje cez všetky ponuky, porovnáva počty hlasov a určí

víťaznú ponuku na základe najvyššieho počtu hlasov. Podmienkou úspešnej finalizácie je, že aspoň jedna ponuka obdržala aspoň jeden hlas. Po finalizácii sa tender presunie do stavu `Completed` a emituje sa udalosť `TenderCompleted` s identifikátorom víťaznej ponuky. Výsledky hlasovania je možné následne overiť prostredníctvom funkcie `getWinningBid()`, ktorá vráti identifikátor víťaznej ponuky, adresu dodávateľa, cenu a počet hlasov.

Tvorca tendra môže následne administratívne označiť tender ako splnený prostredníctvom `fulfillTender()`, čím sa stav zmení na `Fulfilled`.




Obrázok 4.5: Hlasovanie a ukončenie tendra.

Je potrebné zdôrazniť, že aktuálna verzia riešenia nepokrýva automatizované on-chain platby, escrow mechanizmus ani riešenie sporov. Samotné plnenie zákazky a finančné vyrovnanie prebiehajú mimo blockchain, čo zodpovedá reálnej praxi mikro-tendrov v študentských organizáciách. Návrh sa sústreďuje na transparentnú evidenciu, správu ponúk a demokratické hlasovanie, čo je pre rozsah bakalárskej práce primerané.

## 4.4 Návrh integrácie s IPFS

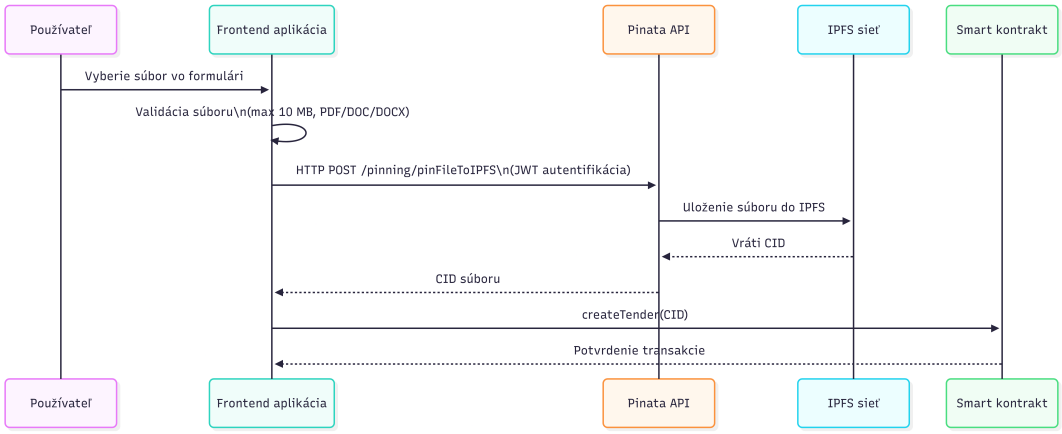
Pre ukladanie dokumentácie k tendrom bol zvolený systém IPFS (InterPlanetary File System). Hlavným dôvodom je nevhodnosť priameho ukladania väčších súborov do blockchainu, kde náklady na úložisko rastú lineárne s veľkosťou dát.

Kým blockchain je určený na nemenné uloženie kľúčových metadát a logiky procesu, IPFS je vhodnejšie pre samotné dokumenty, prílohy a ďalšie rozsiahlejšie súbory.

### 4.4.1 Proces nahrávania cez Pinata

V navrhovanom riešení prebieha nahrávanie dokumentu priamo z frontendu cez službu Pinata, bez potreby samostatného backendového servera. Proces pozostáva z nasledujúcich krokov:

1. Používateľ vo formulári vyberie súbor, ktorý chce priložiť k tendru.
2. Frontend vykoná validáciu: overí, že veľkosť súboru nepresahuje 10 MB a typ súboru zodpovedá povoleným formátom (PDF, DOC, DOCX).
3. Súbor sa odošle na Pinata API prostredníctvom HTTP POST požiadavky na endpoint `/pinning/pinFileToIPFS`. Autentifikácia prebieha prostredníctvom JWT tokenu uloženého v premennej prostredia.
4. Pinata súbor spracuje, uloží do IPFS siete a vráti CID.
5. Frontend obdržaný CID uchová a pri vytváraní tendra ho odovzdá smart kontraktu ako parameter funkcie `createTender()`.




Obrázok 4.6: Proces nahrávania dokumentu do IPFS.

Takto navrhnutý postup má dve dôležité vlastnosti. Po prvé, nie je potrebné prevádzkovať samostatný backend len na účely uploadu súborov – celá komunikácia s IPFS prebieha na strane klienta. Po druhé, používateľ vníma celý proces ako súčasť jednej aplikácie, hoci technicky ide o kombináciu komunikácie s IPFS a blockchainom.

### 4.4.2 Prístup k dokumentom

Po uložení CID do kontraktu je možné dokument získať späť prostredníctvom IPFS gateway. Frontend preto nepotrebuje uchovávať vlastnú kópiu dokumentu ani ho ukladať do ďalšieho úložiska. V detaile tendra môže používateľ dokument otvoriť alebo stiahnuť prostredníctvom URL adresy odvodenej z CID, napríklad vo formáte `https://ipfs.io/ipfs/{CID}`.

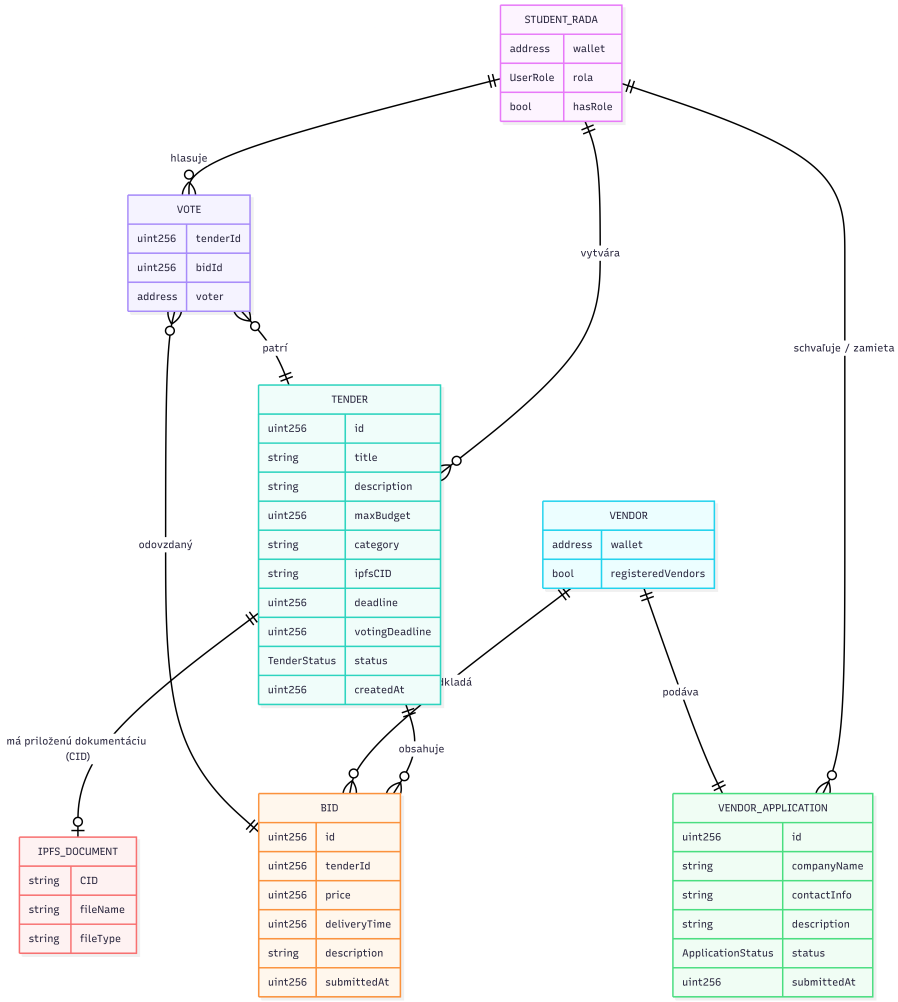
Aplikácia používa utilitu `getIPFSUrl()`, ktorá transformuje CID na prístupovú URL adresu. Tento prístup je transparentný a univerzálny – akýkoľvek používateľ s prístupom k CID môže dokument overiť nezávisle na frontende aplikácie, prostredníctvom ľubovoľného IPFS klienta alebo verejného gateway.

## 4.5 Návrh používateľského rozhrania (UI/UX)

Používateľské rozhranie bolo navrhnuté v minimalistickom štýle, aby používatelia rýchlo pochopili, ako webovú stránku používať. Všetky nižšie uvedené informácie sú k dispozícii v projekte na webovej stránke Figma.

### 4.5.1 UX výskum a konceptuálny model

Návrh UI bol spracovaný iteratívne na základe UX výskumu a prototypovania. Výskum definoval dva primárne používateľské typy: člen študentskej rady a dodávateľ. Z pohľadu funkcionality bolo potrebné pokryť štyri jadrové scenáre: vytvorenie tendra, správu ponúk, hlasovanie a zobrazenie výsledkov.



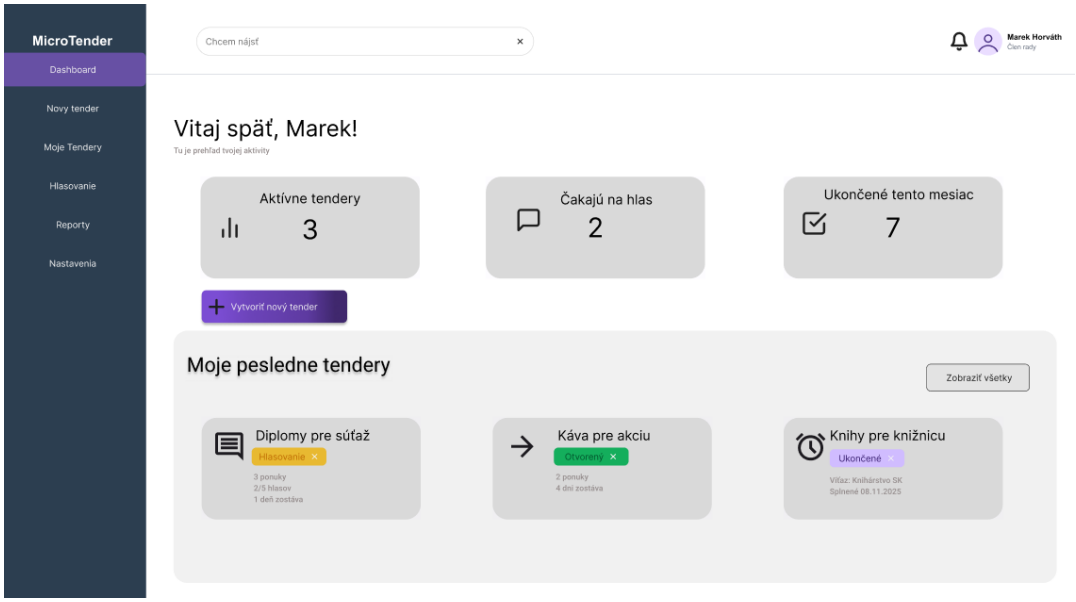

Obrázok 4.7: Konceptuálny model používateľského rozhrania.

### 4.5.2 First prototype

Prvý prototyp (obr. 4.7) bol vytvorený v nástroji Figma ako low-fidelity návrh. Cieľom bolo overiť základnú štruktúru aplikácie, rozloženie hlavných prvkov a zrozumiteľnosť navigácie pred samotným detailným dizajnom a implementáciou. Prototyp[9] zahŕňal kľúčové obrazovky pre obe používateľské roly: dashboard s prehľadom tenderov, formulár na vytvorenie tenderu, detail tenderu s možnosťou podania ponuky a proces hlasovania[10].

[9]Odkaz na bežiaci prototyp: https://www.figma.com/proto/zbuCAJYvxQK20s2klfxobp/UX-Kostiantyn-Ostrovskyi---Prototype?node-id=31-1140

[10]Odkaz na súbor prototypu https://www.figma.com/design/zbuCAJYvxQK20s2klfxobp/UX-Kostiantyn-Ostrovskyi---Prototype?node-id=0-1&p=f&t=dzIvgVTuzXryBcaH-0




Obrázok 4.8: Ukážka z prvého prototypu dashboardu.

### 4.5.3 Používateľské testovanie prvého prototypu

Testovanie prebehlo na vzorke 5 participantov. Išlo o skupinu študentov technického zamerania vo veku 17-19 rokov, všetci muži s pokročilými znalosťami práce s počítačom. Každý participant mal za úlohu prejsť dvoma hlavnými scenármi: (1) vytvorenie nového tenderu ako člen študentskej rady, (2) ohodnotiť a hlasovať za jednu z troch predpripravených ponúk v rámci iného existujúceho tenderu[11]. Priebeh testovania bol zaznamenaný na video a následne každý participant vyplnil štandardizovaný dotazník SUS[12].

**Výsledky:** Priemerné SUS skóre prvého prototypu dosiahlo hodnotu **90.5 bodov**, čo podľa štandardnej interpretácie Bangor et al. (2009) predstavuje hodnotenie **A+ (výnimočné)**. Individuálne skóre participantov sa pohybovalo v rozmedzí 72.5 až 100.0 bodov, pričom štyria z piatich účastníkov dosiahli skóre nad 90 bodov.

**Zistené nedostatky:** Na základe analýzy videozáznamov a slovnej spätnej väzby boli identifikované hlavné problémové oblasti:

- Formulár na vytvorenie tenderu bol príliš dlhý a zahltil používateľov množstvom polí naraz.

[11]Odkaz na sekciu používateľského testovania: https://www.figma.com/board/lB1lqKaJyCjK50MMjoj7eB/UX-3-Kostiantyn-Ostrovskyi?node-id=2011-333

[12]System Usability Scale (SUS). Výsledky: https://docs.google.com/spreadsheets/d/19ulSl-Qp4TIzzSMI0Paw5tcCaPp7P8a5wViKzp4I29M/edit?gid=774672262

- Tlačidlá mali nedostatočný vizuálny kontrast, čo spôsobovalo váhanie pri výbere akcií.
- Dizajn bol funkčný, ale vizuálne nepôsobil profesionálne a požadoval estetické vylepšenie.

Tieto nedostatky boli v druhej iterácii odstránené.

**Faktory ovplyvnenia:** Keďže prvý prototyp bol skôr pracovnou vzorkou než detailne spracovaným dizajnom, niekoľko participantov vyjadrilo výhrady k vizuálnej stránke. Napriek tomu vysoké SUS skóre potvrdzuje, že základná funkcionalita a navigácia boli intuitívne.

### 4.5.4 Druhý prototyp a jeho overenie

Na základe výsledkov používateľského testovania prvého prototypu a identifikovaných nedostatkov bola vypracovaná druhá iterácia návrhu. Cieľom bolo nielen odstrániť konkrétne problémy zistené počas testovania, ale aj posunúť celkový vizuálny štýl aplikácie na profesionálnejšiu úroveň. Druhý prototyp predstavuje finálny hi-fi návrh rozhrania, ktorý bol následne implementovaný v rámci vývojovej fázy projektu.

### 4.5.5 Zmeny a vylepšenia oproti prvému prototypu

V porovnaní s prvou iteráciou boli vykonané tri hlavné zmeny.

**Prepracovanie vizuálneho štýlu.** Celkový vzhľad aplikácie bol zrevidovaný s dôrazom na modernú a profesionálnu estetiku. Farebná paleta, typografia a rozmiestnenie prvkov boli zjednotené do konzistentného dizajnového systému, čím sa zvýšila vizuálna dôveryhodnosť aplikácie.

**Vylepšenie hlasovacej obrazovky.** Tlačidlá na hlasovacej stránke boli prepracované so zlepšeným vizuálnym kontrastom a jasnejšou hierarchiou akcií. Táto zmena priamo reaguje na zistenie z testovania prvého prototypu, kde participanti váhali pri výbere správnej akcie.

**Prepracovanie obrazovky Moje tendre.** Obrazovka zobrazujúca tendre vytvorené prihláseným členom rady bola kompletne redesignovaná. Nové rozloženie informácií je prehľadnejšie a umožňuje rýchlejšiu orientáciu v zozname vlastných tendrov.

### 4.5.6 Popis finálneho rozhrania

Rozhranie je organizované ako jednostránková aplikácia s bočnou navigáciou (`Sidebar`) a záhlavím (`Header`). Navigačné položky a dostupné akcie sa dynamicky prispôsobujú

roli prihláseného používateľa načítanej zo smart kontraktu. Návštevník bez pripojenej peňaženky môže prezerať verejné informácie o tendroch v režime len na čítanie, čo zabezpečuje transparentnosť systému pre širokú verejnosť.

Aplikácia pokrýva nasledujúce hlavné obrazovky:

- **Dashboard** – celkový prehľad systému so súhrnnými štatistikami a zoznamom najnovších tendrov.
- **Všetky tendre / Moje tendre** – zoznamy tendrov filtrované podľa kontextu používateľa.
- **Vytvorenie tendra** – dvojkrokový formulár pre vytvorenie konceptu a následné publikovanie tendra s možnosťou priloženia dokumentácie.
- **Detail tendra** – kompletné informácie o tendri, zoznam predložených ponúk a možnosť hlasovania alebo podania ponuky podľa roly používateľa.
- **Hlasovanie** – prehľad tendrov vo fáze hlasovania s prehľadne zobrazenými ponukami a výsledkami.
- **Registrácia dodávateľa / Schvaľovanie žiadostí** – workflow pre registráciu externých subjektov a ich správu členom rady.

Pri každej operácii meniacej stav blockchainu aplikácia zobrazuje indikátor spracovania transakcie a po jej dokončení informuje používateľa o výsledku. Rozhranie je responzívne – na menších displejoch sa bočné menu nahradí výsuvným panelom, čím sa zachováva použiteľnosť na mobilných zariadeniach.

### 4.5.7 Používateľské testovanie druhého prototypu

Druhý prototyp bol overený používateľským testovaním na rovnakej vzorke participantov ako prvá iterácia. Testovacie scenáre zostali totožné, čo umožňuje priame porovnanie výsledkov medzi oboma iteráciami. Výsledky testovania vrátane SUS skóre sú uvedené nižšie.

(*Výsledky testovania budú doplnené po vyhodnotení dotazníkov.*)

# 5 Vyhodnotenie

Vyhodnocovacia časť je kľúčovou časťou záverečnej práce. Tato časť obsahuje vyhodnotenie navrhnutého (vytvoreného) riešenia. Uprednostňované je objektívne vyhodnotenie výsledkov práce, ktoré sa opiera o meranie a štatistické metódy, prípadne matematické dôkazy. V prípade nameraných hodnôt musí autor opísať metódu merania, priebeh merania, výsledky a interpretáciu výsledkov v kontexte riešeného problému a stanovených cieľov. Na základe vyhodnotenia riešenia autor opíše prínosy svojej práce. Vyhodnocovacia časť tvorí zvyčajne ¼ jadra práce.

# 6 Záver

Záver práce obsahuje zhrnutie výsledkov práce s jasným opisom prínosov a pôvodných (vlastných) výsledkov autora a vyhodnotenie splnenia stanovených cieľov. Je to stručné zhrnutie informácií uvedených v záverečnej práci. Záver by nemal obsahovať nové informácie.

V závere by mal tiež autor poukázať na prípadné otvorené otázky, ktoré sú nad rámec rozsahu práce a mal by odporučiť ďalšie aktivity na pokračovanie pri riešení problému. Rozsah záveru je minimálne 1 celá strana.

# Zoznam príloh

**Príloha A** Používateľská príručka

**Príloha B** Systémová príručka

**Príloha C** Štruktúra záverečnej práce

**Príloha D** LaTeX 101

**Príloha E** CD médium – záverečná práca v elektronickej podobe:

- `doc` – záverečná práca a prílohy
- `tex` – zdrojové súbory záverečnej práce a vedeckého článku
- `src` – zdrojové kódy vytvoreného riešenia

# A Používateľská príručka

Toto je príklad používateľskej príručky, ktorá môže byť súčasťou záverečnej práce. Obsahuje základné informácie o tom, ako používať vytvorený systém alebo aplikáciu. Príručka by mala byť zrozumiteľná a poskytovať jasné pokyny pre používateľov.

## 1 Inštalácia

# B Systémová príručka

# C Štruktúra záverečnej práce

Na písanie záverečných prác je možné použiť viacero štruktúr. Pre vytvorenie tejto práce sme použili dve z nich:

1. hlavná štruktúra práce bola vytvorená podľa štruktúry uvedenej v článku [**Beel2010**]
2. pre vytvorene abstraktu bola použitá štruktúra [**WikipediaImrad**], ktorá môže byť použitá na napísanie celej práce

Krátky význam jednotlivých kapitol bude uvedený v nasledujúcom texte. Ďalšie užitočné informácie nájsdete v *Pokynoch pre vypracovanie záverečných prác*[13].

## Abstrakt

Abstrakt by mal byť dlhý 100 až 300 slov. Pre jeho organizáciu môžete použiť štruktúru *IMRaD*, čo je skratka pre:

- **Introduction** - jasne predstavte zámer svojej práce, použite prítomný alebo minulý čas
- **Methods** - uveďte použité výskumné metódy, použite minulý čas
- **Results** - zhrňte hlavné dosiahnuté výsledky, použite prítomný alebo minulý čas
- **Discussion** - na záver zhrňte hlavné závery vyplývajúce z vašej práce, použite prítomný čas

Na základe uvedenej štruktúry môžete abstrakt napísať tak, že každý bod napíšete v jednej, ale maximálne v dvoch vetách. Určite však do abstraktu neprepisujte obsah práce! Snažte sa, aby **rozsah abstraktu spolu s bibliografickou citáciou mal práve jednu stranu**!

[13] https://theses.kpi.fei.tuke.sk/instructions

## Predhovor

Predhovor ponúka neformálny priestor na vlastné vyjadrenie autora. Môže obsahovať informácie o tom, čo sa autor pri písaní práce naučil, o vlastnej motivácii k výberu témy, o prekonaných výzvach počas písania práce, a pod. Píše sa v prvej osobe jednotného čísla a rozsah by nemal presiahnuť 2 strany.

Táto kapitola nie je povinná!

## Úvod

Úvod práce stručne opisuje stanovený problém, kontext problému a motiváciu pre jeho riešenie. Z úvodu by malo byť jasné, že stanovený problém má zmysel riešiť.

V úvode neuvádzajte štruktúru práce! Rozsah tejto kapitoly by mal byť minimálne 2 strany.

Súčasťou úvodu je aj *formulácia úlohy*. Vo formulácii úlohy uveďte jasne a vecne ciele práce. Čím budú ciele konkrétnejšie, tým bude jasnejšie, aký problém riešite a ako ho chcete riešiť. Formuláciu úlohy vytvorte po dostatočnom naštudovaní problematiky a vytvorení analytickej časti.

## Analytická časť

Analytická časť záverečnej práce analyzuje existujúce podobné prístupy k riešeniu stanoveného problému. Autor práce musí uviesť v tejto časti existujúce prístupy a riešenia, pričom musí zaujať stanovisko k týmto prístupom a riešeniam a opísať ich výhody a nedostatky. Prevažne v tejto časti autor používa odkazy na použité zdroje. Autor v analýze nepreberá odseky z cudzích prác ale uvádza prevažne vlastné postoje podložené odkazmi na literatúru. Analytická časť práce by teda nemala byť len povrchným prepisom základných informácií z Wikipédie alebo zo stránok opisovaných nástrojov. Je potrebné aby bola analýza podporená aj experimentmi ak to umožňuje téma práce (napr. vyskúšam softvér). Vďaka popisu existujúcich riešení autor pochopí problematiku, viac sa nad riešeniami zamyslí, usporiada si ich, zistí ich kladné a záporné vlastnosti, z čoho potom postupne vyplynie návrh vlastného riešenia v syntetickej časti. Analytická časť tvorí zvyčajne ¼ jadra práce.

Analytickú časť je vhodné rozdeliť na niekoľko kapitol, ktoré budú venované rôznym analyzovaným témam. Názvy kapitol majú zodpovedať tomu, čo je v kapitole

opisované. Napríklad, ak v práci analyzujete súčasný stav v oblasti medzigalaktických letov, namiesto všeobecného názvu „Analýza súčasného stavu“ by mal byť použitý názov analyzovanej témy – „Medzigalaktické lety“.

Mali by ste v práci mať minimálne kapitoly opisujúce

- doménu (oblasť) riešeného problému a jej súčasný stav,
- súvisiace práce a existujúce riešenia.

Je veľmi pravdepodobné, že sa daný problém pokúšal vyriešiť už aj niekto iný. Alebo existujú riešenia podobné tomu, ktoré potrebujete vyriešiť vy. V tejto časti práce teda popíšete, aké riešenia už existujú a aké sú ich výhody a nevýhody. Týmto spôsobom preukážete, že ste si naštudovali tému a vyskúšali rôzne riešenia, a môžete sa inšpirovať už existujúcimi riešeniami.

## Ciele záverečnej práce

Vychádzajúc z analýzy problému a existujúcich riešení by ste mali byť schopní stanoviť konkrétne implementačné ciele vašej práce. Tie môžete uviesť v samostatnej kapitole, napríklad vo forme zoznamu alebo odrážok. Rozhodne tu neprepisujte zadávací list, ale jasne a stručne uveďte, svoje čiastkové ciele. Neskôr ich budete vedieť v kapitole *Vyhodnotenie* vyhodnotiť aj s odôvodnením, či sa vám uvedené ciele splniť podarilo alebo nie.

## Syntetická časť

Syntetická časť opisuje metódy použité na syntézu riešenia a opisuje syntézu samotného riešenia (zvyčajne je to návrh/implementácia softvérového resp. hardvérového riešenia), pričom sa opiera o závery analytickej časti práce. Začína od toho, ako sa bude riešenie používať: najdôležitejšie scenáre používania a používateľské rozhranie, ktoré bude tieto scenáre efektívne podporovať. Až potom je na rade vnútorná architektúra alebo použité technológie. Syntetická časť tvorí zvyčajne ½ jadra práce.

Syntetickú časť práce vhodne rozdeľte do kapitol a pomenujte ich podľa toho, čomu sú venované. Podľa obsahu práce môže táto časť obsahovať kapitoly opisujúce, napríklad:

- použitú metodológiu – metodológia experimentu alebo spôsob riešenia úlohy,
- návrh riešenia – celkový opis riešenia a jeho zdôvodnenie,

- implementáciu – najpodstatnejšie implementačné rozhodnutia a ich zdôvodnenie.

## Vyhodnotenie

Vyhodnocovacia časť je kľúčovou časťou záverečnej práce. Tato časť obsahuje vyhodnotenie navrhnutého (vytvoreného) riešenia. Uprednostňované je objektívne vyhodnotenie výsledkov práce, ktoré sa opiera o meranie a štatistické metódy, prípadne matematické dôkazy. V prípade nameraných hodnôt musí autor opísať metódu merania, priebeh merania, výsledky a interpretáciu výsledkov v kontexte riešeného problému a stanovených cieľov. Na základe vyhodnotenia riešenia autor opíše prínosy svojej práce. Vyhodnocovacia časť tvorí zvyčajne ¼ jadra práce.

Okrem iného môže byť súčasťou vyhodnotenia:

- zhrnutie dosiahnutých výsledkov,
- interpretácia výsledkov,
- objektívne overenie dosiahnutia cieľov práce

## Záver

Záver práce obsahuje zhrnutie výsledkov práce s jasným opisom prínosov a pôvodných (vlastných) výsledkov autora a vyhodnotenie splnenia stanovených cieľov. Je to stručné zhrnutie informácií uvedených v záverečnej práci. Záver by nemal obsahovať nové informácie.

V závere by mal tiež autor poukázať na prípadné otvorené otázky, ktoré sú nad rámec rozsahu práce a mal by odporučiť ďalšie aktivity na pokračovanie pri riešení problému. Rozsah záveru je minimálne 1 celá strana.

## Prílohy

Prílohy môžu obsahovať doplňujúce materiály, ktoré sú dôležité pre pochopenie práce, ale nie sú nevyhnutné pre čitateľa, aby mohol pochopiť hlavný obsah práce. Prílohy majú obsahovať aj príručku pre používateľa a systémovú príručku pre nasadenie a pokračovanie vo vývoji.

# D LaTeX 101

## 1 Základy písania

Odseky textu oddeľujte jednoducho prázdnym riadkom. To, že prvý odsek v kapitole nie je odsadený, je štandardný typografický postup a nie je potrebné ho meniť.

Časť textu môžeme *mierne zvýrazniť* pomocou príkazu `\emph{}`. Prípadne použiť **tučné písmo** príkazom `\textbf{}`.

Na vytvorenie zoznamu sa používa prostredie `itemize`:

- raz,
- dva,
- tri.

Zoznam môže byť aj číslovaný ak vymeníme `itemize` za `enumerate`:

1. raz,
2. dva,
3. tri.

## 2 Členenie textu

Na definovanie kapitol a podkapitol sa používajú príkazy

- `\chapter{}`,
- `\section{}`,
- `\subsection{}`.

Hlbšie úrovne vnorenia sa neodporúča používať. Tak isto neodporúčame mnohonásobne vnorené zoznamy.

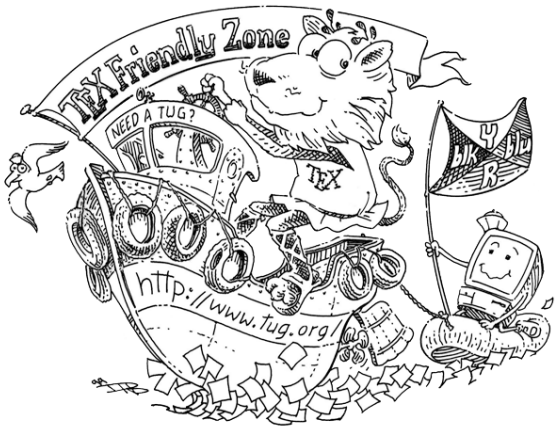
Ak za príkaz pridáte hviezdičku kapitola nebude číslovaná a ani sa nezobrazí v obsahu. Neodporúčame to však používať mimo príloh.

# 3 Obrázky

Na vkladanie obrázkov sa používa prostredie `figure`:

```
\begin{figure}
  \centering
  \includegraphics[width=0.5\textwidth]{figures/tugboat}
  \caption{\LaTeX{} Friendly Zone \label{o:latex_friendly_zone}}
\end{figure}
```

Výsledkom bude obrázok 1.




Obrázok 1: LaTeX Friendly Zone

Na samotné vloženie obrázka sa používa príkaz `\includegraphics{}`. LaTeX podporuje bežné formáty ako PNG a JPEG. Pre vektorovú grafiku je vhodné použiť formát PDF.

Každý obrázok by mal mať popis, ktorý je uvedený v *caption*. A čo je veľmi dôležité, na každý obrázok by mal byť odkaz v texte. Na to použite príkazy `\label{}` a `\ref{}`. Prvý definuje názov, ktorým sa na obrázok odkazujete, druhý vytvorí odkaz na obrázok. Napríklad, obrázok 1 zobrazuje prostredie priateľské pre používateľov LaTeX-u.

LaTeX má vstavené pravidlá pre umiestnenie obrázkov. Štandardne ich umiestni na vrch stránky, na ktorej sú definované, aby text nebol prerušený. Môžete však použiť voliteľné parametre, aby ste ovplyvnili umiestnenie obrázka. Napríklad `[!ht]` znamená, že obrázok sa má, ak je to možné, umiestniť presne tam, kde je definovaný, inak na vrchu stránky:

```
\begin{figure}[!ht]
```

# 4 Tabuľky

Tabuľky sa vkladajú do prostredí `table` a `tabular`

Tabuľka 1: Kódy krajín podľa štandardov ISO

| Názov krajiny | Alpha 2 | Alpha 3 | Numeric |
|---|---|---|---|
| Afghanistan | AF | AFG | 004 |
| Alandské Ostrovy | AX | ALA | 248 |
| Albánsko | AL | ALB | 008 |
| Alžírsko | DZ | DZA | 012 |
| Americká Samoa | AS | ASM | 016 |
| Andorra | AD | AND | 020 |
| Angola | AO | AGO | 024 |

Pre sadzbu profesionálne vyzerajúcich tabuliek odporúčame použiť balík *booktabs*[14].

# 5 Výpisy kódu

Pre výpisy kódu sa používa prostredie `lstlisting`:

Výpis 1: Program, ktorý pozdraví celý svet

```
#include <stdio.h>
int main() {
    /* Print Hello, World! */
    printf("Hello, World!\n");
    return 0;
}
```

Výpisy môžu mať voliteľný nadpis, ktorý sa zobrazí nad výpisom. Tak isto je možné im definovať `label`, na ktorý sa môžete odkazovať (viď výpis 1).

Obsah výpisu môže byť tiež načítaný zo súboru pomocou príkazu:

Výpis 2: Riešenie problému Schody

```
#include <karel.h>

// function definition
void turn_right() {
```

[14] https://en.wikibooks.org/wiki/LaTeX/Tables#Professional_tables

```
    set_step_delay(0);
    turn_left();
    turn_left();
    set_step_delay(400);
    turn_left();
}

int main() {
    turn_on("stairs1.kw");

    set_step_delay(400);

    pick_beeper();
    turn_left();
    step();
    // function call
    turn_right();
    step();

    turn_off();
}
```

# 6 Citácie

Na citovanie literatúry sa používa balík *biblatex*. Citácie sa vkladajú pomocou príkazu `\cite{}`. Napríklad, [**Beel2010**] je článok, ktorý popisuje štruktúru záverečnej práce.

Bibliografické záznamy sú definované v súbore `bibliography.bib`. Každý záznam má unikátny identifikátor, ktorý sa používa na citovanie. Napríklad, záznam `Beel2010` vyzerá nasledovne:

```
@online{Beel2010,
  title  = {How to write a thesis (Bachelor, Master, or PhD) and ↩
     which software tools to use},
  url    = {https://isg.beel.org/blog/2010/03/02/how-to-write-a-↩
     phd-thesis/},
  author = {Joeran Beel},
  year   = {2010},
  urldate = {2024-09-26}
```

```
}
```

Za znakom `@` je uvedený typ záznamu, v tomto prípade `online`. Iné typy záznamov môžu byť napríklad `article`, `book`, `inproceedings`, a pod. Každý záznam má povinné a nepovinné položky. Pre viac informácií o formáte záznamov v bibliografickom súbore odporúčame pozrieť stránku *The 14 BibTeX entry types*[15].

Pri online zdrojoch nezabudnite uviesť dátum prístupu v položke `urldate`, keďže také zdroje sa môžu časom meniť.

[15] https://www.bibtex.com/e/entry-types/